

Konzeption und Evaluation eines domänenbasierten Explorers als VS- Code Extension für SAP CAP Anwendungen

Bachelorarbeit

aus dem Studiengang Wirtschaftsinformatik Software Engineering

an der Dualen Hochschule Baden-Württemberg Mannheim

von

Max Christian Meinel

11.05.2026

Matrikelnummer, Kurs:	7864687, WWI23SEB
Unternehmen:	SAP SE, 69190, Walldorf
Abteilung:	CAP Tools & MTX
Leiter des Studiengangs:	Prof. Dr. Henning Pagnia
Betreuer im Unternehmen:	Christian Fuchs
Betreuer an der DHBW:	Ulrich Wolf

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Konzeption und Evaluation eines domänenbasierten Explorers als VS-Code Extension für SAP CAP Anwendungen

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Walldorf 11.05.2026

Max Christian Meinel

Abstract

Die vorliegende Bachelorarbeit von Max Christian Meinel (Kurs WWI23SEB, DHBW Mannheim) entsteht in Kooperation mit SAP SE.

Diese Arbeit untersucht das Navigationsproblem in SAP CAP-Projekten und entwickelt eine VS Code Extension zur servicezentrierten Darstellung verteilter Artefakte. SAP Cloud Application Programming Model strukturiert Projekte nach dem Prinzip der Separation of Concerns in technische Ordner, wodurch ein einzelner fachlicher Service sich über viele Dateien verteilt. Entwickler müssen zur Analyse eines Services zwischen mehreren Dateien navigieren und die Service-Struktur manuell rekonstruieren, was zu hohem Navigationsaufwand führt.

Der gewählte methodische Rahmen ist Design Science Research nach Peffers et al. Die Problemidentifikation quantifiziert den Navigationsaufwand anhand repräsentativer Szenarien. Das komplexeste Szenario erfordert zehn Navigationsschritte über drei Dateien. Eine Nutzwertanalyse vergleicht drei Integrationsstrategien und identifiziert die Language-Server-basierte Integration als vorteilhafteste Lösung. Die Extension kommuniziert mit dem CAP Language Server über das Language Server Protocol, transformiert die erhaltenen Symboldaten in eine hierarchische Baumstruktur und visualisiert diese über die TreeView API.

Die Evaluation testet die Extension an drei Projekten unterschiedlicher Größe. Die Extension erfüllt sechs von acht Anforderungen vollständig, darunter die automatische Service-Identifikation, aggregierte Darstellung aller Artefakte und präzise Navigation zu Quellcode-Positionen. Die Messungen zeigen eine Reduktion des Navigationsaufwands um durchschnittlich 79 Prozent in den evaluierten Szenarien. Die servicezentrierte Sicht ergänzt die dateibasierte Navigation und reduziert den kognitiven Aufwand beim Verstehen und Bearbeiten von Services.

Inhaltsverzeichnis

1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.3 Zielsetzung der Arbeit	2
1.4 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 SAP Cloud Application Programming Model (CAP)	4
2.2 Language Server und Language Server Protocol	8
2.3 VS Code Extension Architektur	10
2.4 Navigation und Informationssuche in Softwaresystemen	11
3 Methodik	13
3.1 Design Science Research als methodischer Rahmen	13
3.2 Eignung von DSR für diese Arbeit	14
3.3 Auswahl des DSR-Prozessmodells	14
3.4 Anwendung des DSR-Prozesses in dieser Arbeit	16
4 Problem- und Anforderungsanalyse	17
4.1 Problemkontext und Analyse von CAP-Repositories	17
4.2 Identifizierte Herausforderungen bei der Navigation	21
4.3 Ableitung der Designziele	25
4.4 Funktionale Anforderungen	25
4.5 Nicht-funktionale Anforderungen	26
4.6 Abgrenzung der Arbeit	27
5 Integrationsstrategien	28
5.1 Repository-basierte Integration	28
5.2 CSN-basierte Integration	31
5.3 Language-Server-basierte Integration	34
5.4 Vergleich und Auswahl	37
5.5 Auswahl der Integrationsstrategie	40
6 Implementierung des Prototyps	42
6.1 Architekturkonzept und Designentscheidungen	42
6.2 Integration mit CDS Language Server	44
6.3 Datenmodell und Baumaufbau	46

6.4 Navigation und Interaktion	49
6.5 Fehlerbehandlung und Benutzer-Feedback	50
6.6 Ergebnis der Implementierung	52
7 Evaluation	55
7.1 Testprojekte	55
7.2 Vergleich mit bestehenden Ansätzen	56
7.3 Ergebnisse der Evaluation	57
7.4 Explorative Entwickler-Rückmeldungen	59
7.5 Qualitative Beobachtungen	60
7.6 Einordnung der Ergebnisse	61
8 Diskussion und Fazit	62
8.1 Diskussion der Ergebnisse	62
8.2 Wissenschaftlicher Beitrag	62
8.3 Limitationen	63
8.4 Praktische Implikationen	64
8.5 Ausblick	65
8.6 Reflexion der Forschungsmethode	65
8.7 Fazit	66
Literatur	a

Abbildungsverzeichnis

Abbildung 1	CAP-Architektur mit CDS als zentralem Baustein [1]	5
Abbildung 2	CDS-Kompilierungsfluss mit CSN als zentralem Zwischenformat [2]	6
Abbildung 3	Konzeptionelle Fragmentierung eines CAP-Services (vereinfacht).	20
Abbildung 4	Navigationskomplexität typischer Entwicklungsaufgaben im Bookshop-Projekt	24
Abbildung 5	Ergebnis der Nutzwertanalyse	41
Abbildung 6	Komponentenstruktur und Datenfluss der VS Code Extension . .	43
Abbildung 7	Navigationsstruktur der Extension [3]	48
Abbildung 8	Extension mit TreeView und Code-Navigation [3]	52
Abbildung 9	Suchfunktion mit gefilterter Baumansicht [3]	53
Abbildung 10	Filterung nach Artefakt-Typen [3]	54
Abbildung 11	Ladezeiten der Extension nach Projektöffnung	58

Tabellenverzeichnis

Tabelle 1	Zentrale CDS-Artefakt-Typen mit Notation und Verwendungskontext	7
Tabelle 2	Zentrale Artefakte eines CAP-Services (basierend auf [4, Sec. Domain Modeling] Authorization Annotations: [5, Sec. Role-Based Access Control])	21
Tabelle 3	Szenario „Service verstehen“ im Bookshop-Projekt	23
Tabelle 4	Bewertungskriterien und Gewichtung	38
Tabelle 5	Nutzwertanalyse der Integrationsstrategien	40
Tabelle 6	Error-Typen und deren Behandlung in der Extension	50
Tabelle 7	Charakterisierung der Testprojekte nach Größenmetriken	55
Tabelle 8	Vergleich des Navigationsaufwands	59
Tabelle 9	Übersicht der Anforderungserfüllung	59
Tabelle 10	Testumgebung für Performance-Messungen	g
Tabelle 11	Flags für servicezentrierte Navigation	i
Tabelle 12	Übersicht der explorativen Entwickler-Rückmeldungen	j
Tabelle 13	Detaillierte Begründung der Punktvergabe in der Nutzwertanalyse ..	k
Tabelle 14	Szenario „Feld hinzufügen“ – 6 Dateien, 6 Navigationsschritte	l
Tabelle 15	Szenario „Auth anpassen“ – 3 Dateien, 4 Navigationsschritte	l
Tabelle 16	Szenario „Handler debuggen“ – 3 Dateien, 4 Navigationsschritte	l
Tabelle 17	Navigationsablauf „Service verstehen“ (xtravels) – Vergleich	m
Tabelle 18	Navigationsablauf „Service verstehen“ (bookshop) – Vergleich	n
Tabelle 19	Navigationsablauf „Service verstehen“ (ctsm-develop) – Vergleich ...	o
Tabelle 20	Navigationsablauf „Action lokalisieren“ (xtravels) – Vergleich	p

Tabelle 21	Navigationsablauf „Action lokalisieren“ (bookshop) – Vergleich	q
Tabelle 22	Navigationsablauf „Entity lokalisieren“ (ctsm-develop) – Vergleich	r
Tabelle 23	Navigationsablauf „Exponierte Entities identifizieren“ (xtravels) – Vergleich	s
Tabelle 24	Navigationsablauf „Exponierte Entities identifizieren“ (bookshop) – Vergleich	t
Tabelle 25	Navigationsablauf „Exponierte Entities identifizieren“ (ctsm-develop) – Vergleich	u

Codeverzeichnis

Abbildung 1	Exemplarische Struktur eines SAP CAP-Projekts (vereinfacht) [6]. .	18
Quellcode 2	CDS-Service-Definition mit Projections und Action [6]	29
Quellcode 3	Ausschnitt eines CSN-Modells mit Service und Entity	32
Quellcode 4	Beispiel einer Workspace-Symbol-Antwort des CAP Language Servers	35
Quellcode 5	TreeNode Interface	47
Quellcode 6	LSP-Response für ein Symbol aus der /gax Query	h

Glossar

Abkürzungen

API	Application Programming Interface
AST	Abstract Syntax Tree
CAP	Cloud Application Programming Model
CDL	CDS Definition Language
CDS	Core Data Services
CSN	Core Schema Notation
CSS	Cascading Style Sheets
DSR	Design Science Research
HTML	Hypertext Markup Language
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LSP	Language Server Protocol
SDK	Software Development Kit
SaaS	Software as a Service
UI	User Interface
URI	Uniform Resource Identifier
VSIX	Visual Studio Extension

Fachbegriffe

AsyncAPI	Spezifikation zur Beschreibung ereignisgesteuerter APIs, ähnlich zu OpenAPI für REST-APIs
Composition	Starke Beziehung zwischen zwei Entities in CDS, bei der die abhängige Entity ohne die Haupt-Entity nicht existieren kann

Domain-Driven Design	Softwareentwicklungsansatz, der die fachliche Domäne und deren Modellierung in den Mittelpunkt stellt
Domänenspezifische Sprache	Programmier- oder Modellierungssprache, die für einen spezifischen Anwendungsbereich entwickelt wurde
Enterprise-Anwendung	Geschäftsanwendung für Unternehmen mit hoher Komplexität und Skalierbarkeit
Extension	Erweiterung für VS Code, die zusätzliche Funktionalität bereitstellt
Go-to-Definition	IDE-Funktion, die es ermöglicht, direkt zur Definition eines Symbols im Quellcode zu springen
HANA	High-Performance Analytic Appliance - In-Memory-Datenbanksystem von SAP
JSON-RPC	JSON Remote Procedure Call - Protokoll für entfernte Prozeduraufrufe über JSON
JavaScript	Interpretierte Programmiersprache für client-seitige und serverseitige Webanwendungen, bildet die Grundlage für TypeScript
Kognitive Belastung	Mentale Anstrengung, die beim Verarbeiten von Informationen entsteht, wobei die menschliche Verarbeitungskapazität begrenzt ist
Language Server	Separater Prozess, der Sprachanalyse übernimmt und intelligente IDE-Funktionen wie Autovervollständigung, Fehleranzeige und Go-to-Definition bereitstellt
Marketplace	Plattform zur Veröffentlichung und Distribution von VS Code Extensions
Mehrschichtige Architektur	Softwarearchitektur-Muster, bei dem die Anwendung in logische Schichten unterteilt wird
Nutzwertanalyse	Entscheidungsmethode, bei der verschiedene Lösungsalternativen anhand gewichteter Kriterien bewertet und verglichen werden

OData	Open Data Protocol - standardisiertes REST-basiertes Protokoll für den Zugriff auf und die Manipulation von Daten über HTTP
OpenAPI	Standardisierte Spezifikation zur Beschreibung von REST-APIs, ermöglicht maschinell lesbare API-Dokumentation
Outline-Ansicht	Strukturierte Übersicht aller Symbole und Definitionen innerhalb einer einzelnen Datei in einer IDE
Parser	Programm, das Quellcode analysiert und in eine strukturierte Repräsentation überführt, um die syntaktische Struktur zu verstehen
Performance	Leistungsfähigkeit einer Software hinsichtlich Geschwindigkeit und Reaktionszeit
PostgreSQL	Open-Source relationales Datenbankmanagementsystem mit Unterstützung für SQL und erweiterte Datentypen
Prototyp	Frühe, funktionale Version einer Software, die zentrale Funktionalitäten demonstriert
SAP CAP	SAP Cloud Application Programming Model - Framework von SAP zur Entwicklung von Enterprise-Anwendungen, das auf dem Prinzip der Separation of Concerns basiert
SQLite	Leichtgewichtige, dateibasierte relationale Datenbank, häufig verwendet für Entwicklung und Tests
Separation of Concerns	Architekturprinzip, bei dem verschiedene Aspekte einer Anwendung (z.B. Datenmodell, Services, UI) in separate Module getrennt werden, um Wartbarkeit und Skalierbarkeit zu verbessern
Serviceorientierte Architektur	Architekturprinzip, bei dem Funktionalität über klar definierte APIs zugänglich gemacht wird
Skalierbarkeit	Fähigkeit eines Systems, mit wachsenden Anforderungen effizient umzugehen

TypeScript	Typsichere Programmiersprache, die auf JavaScript aufbaut und statische Typprüfung zur Compile-Zeit bietet
VS Code	Visual Studio Code - populäre, erweiterbare Code-Editor-Plattform von Microsoft
npm	Node Package Manager - Standard-Paketmanager für JavaScript- und TypeScript-Projekte zur Verwaltung von Abhängigkeiten und Build-Skripten

1 Einleitung

Dieses Kapitel führt in das Thema der Arbeit ein und beschreibt die Motivation, Problemstellung sowie Zielsetzung. Darüber hinaus wird der Aufbau der Arbeit dargelegt.

1.1 Motivation

Ein Entwickler öffnet ein SAP CAP-Projekt und soll einen Service verstehen. Die Service-Definition steht in `srv/`, die zugehörigen Entities in `db/`, die Event Handler in separaten Implementierungsdateien. Um den Service vollständig zu verstehen, muss der Entwickler zwischen mehreren Dateien navigieren und separate Kontextwechsel durchführen. Die Quantifizierung dieses Aufwands erfolgt in Kapitel 4.

SAP CAP ist ein Framework zur Entwicklung von Enterprise-Anwendungen, das auf dem Prinzip der Separation of Concerns basiert [7, Sec. Separation of Concerns]. Die Trennung von Domänenmodell (`db/`), Services (`srv/`) und UI (`app/`) folgt etablierten Architekturprinzipien und unterstützt die Wartbarkeit skalierender Projekte [8, p. 38]. Diese Struktur hat jedoch eine Konsequenz. Ein einzelner Service, die zentrale fachliche Einheit, ist kein einzelnes Artefakt, sondern verteilt sich über viele Dateien in verschiedenen Ordnern.

Die Folge ist ein erhöhter Navigationsaufwand. Entwickler müssen die Struktur eines Services aus verschiedenen Dateien manuell rekonstruieren. Jeder Kontextwechsel zwischen Dateien unterbricht den Gedankenfluss und erhöht die Kognitive Belastung. Diese Problematik verschärft sich mit zunehmender Projektgröße. Die Evaluation in Kapitel 7 zeigt, dass in produktiven Projekten mit mehreren Services und verschachtelten Verzeichnissen die Rekonstruktion der Service-Struktur deutlich mehr Aufwand erfordert als in kleinen Beispielprojekten.

1.2 Problemstellung

Moderne Entwicklungsumgebungen wie VS Code bieten Navigationsfunktionen wie Go-to-Definition, Outline-Ansichten und File Explorer [9, Sec. Go to Definition], [10, Sec. Outline View]. Diese Werkzeuge operieren jedoch datei- oder symbolzentriert, nicht servicezentriert. Eine aggregierte Sicht, die alle zu einem Service gehörenden Artefakte in einer einheitlichen Übersicht bündelt, existiert nicht.

Ein CAP-Service besteht aus mehreren konzeptionellen Ebenen, die physisch voneinander getrennt sind. Die Service-Identität ergibt sich erst aus dem Zusammenspiel von Datenmodell, API-Schnittstelle, Implementierung und Sicherheitsregeln. Beziehungen zwischen Artefakten sind referenziell, nicht visuell. Entwickler müssen die Struktur eines Services aus verschiedenen Dateien manuell rekonstruieren.

1.3 Zielsetzung der Arbeit

Diese Arbeit entwickelt eine VS Code Extension, die eine servicezentrierte Sicht auf CAP-Projekte bereitstellt. Die Extension aggregiert alle zu einem Service gehörenden Artefakte in einer hierarchischen Baumansicht und ermöglicht direkte Navigation zu den entsprechenden Quellcode-Positionen. Das Ziel ist die Reduktion von Navigations- und Kontextwechseln, um Entwicklern ein schnelleres Verständnis und effizientere Bearbeitung von Services zu ermöglichen.

Die Entwicklung folgt dem DSR Ansatz nach Peffers et al. [11, pp. 52–56]. DSR ist eine Forschungsmethode, die darauf abzielt, Artefakte zur Lösung identifizierter Probleme zu konstruieren und zu evaluieren [12, p. 75]. Die Arbeit durchläuft die sechs Phasen des DSR-Prozesses, nämlich Problemidentifikation, Zieldefinition, Entwurf und Entwicklung, Demonstration, Evaluation und Kommunikation.

Die zentrale Herausforderung liegt in der Gewinnung von Service-Strukturinformationen aus CAP-Projekten. Drei Integrationsstrategien werden entwickelt und mittels Nutzwertanalyse verglichen. Die vorteilhafteste Strategie wird prototypisch implementiert und anhand konkreter Entwicklungsszenarien in CAP-Projekten unterschiedlicher Größe evaluiert.

1.4 Aufbau der Arbeit

Die Arbeit gliedert sich wie folgt. Kapitel 2 vermittelt die technischen Grundlagen zu SAP CAP, CDS, CSN, LSP und VS Code Extension Architektur. Kapitel 3 begründet die Wahl des DSR Ansatzes als methodischen Rahmen und ordnet die Phasen den Kapiteln zu. Kapitel 4 analysiert das Navigationsproblem in CAP-Projekten, quantifiziert den Navigationsaufwand anhand repräsentativer Szenarien und leitet funktionale sowie nicht-funktionale Anforderungen ab. Kapitel 5 entwickelt und vergleicht drei Integrationsstrategien zur Gewinnung von Service-Strukturinformationen mittels Nutzwertanalyse. Kapitel 6 beschreibt die prototypische Implementierung der Extension basierend auf der gewählten Strategie. Kapitel 7 evaluiert die Extension anhand der definierten Anforderungen in drei Testprojekten und quantifiziert die Reduktion des Navigationsaufwands. Kapitel 8 diskutiert die Ergebnisse, benennt Limitationen, gibt einen Ausblick auf zukünftige Arbeiten und fasst die Erkenntnisse zusammen.

2 Grundlagen

Dieses Kapitel vermittelt die technischen Grundlagen zum Verständnis der Arbeit. Es beschreibt das SAP CAP, CDS und CSN. Anschließend werden das LSP sowie die VS Code Extension Architektur erläutert.

2.1 SAP Cloud Application Programming Model (CAP)

Das SAP CAP ist ein Framework für Enterprise-Cloud-Anwendungen von SAP [13]. Das zentrale Ziel ist die Maximierung der Produktivität durch integrierte Best Practices. Entwickler erhalten bewährte Lösungsmuster standardmäßig bereitgestellt, ohne diese manuell implementieren zu müssen. Moderne Cloud-Anwendungen folgen dabei etablierten Architekturprinzipien wie Isolation, Elastizität und lose Kopplung [14, Ch. 3, IDEAL Properties].

Das Framework folgt etablierten Architekturprinzipien. Cloud-native Anwendungen zeichnen sich durch Skalierbarkeit, Resilienz und automatisierte Deployment-Prozesse aus [15]. Domain-Driven Design bedeutet, dass der Fokus auf der Geschäftslogik liegt, während technische Details in den Hintergrund treten. Separation of Concerns strukturiert die Anwendung durch klare Trennung von Domänenmodell (Datenbank), Services und Benutzeroberfläche [8]. Serviceorientierte Architektur bedeutet, dass Funktionalität über klar definierte APIs zugänglich gemacht wird [16].

Abbildung 1 zeigt die Architektur von CAP im Überblick. CDS bilden den zentralen Baustein, der Services, Datenmodelle und UIs verbindet. CAP unterstützt zwei Laufzeitumgebungen: Node.js und Java.

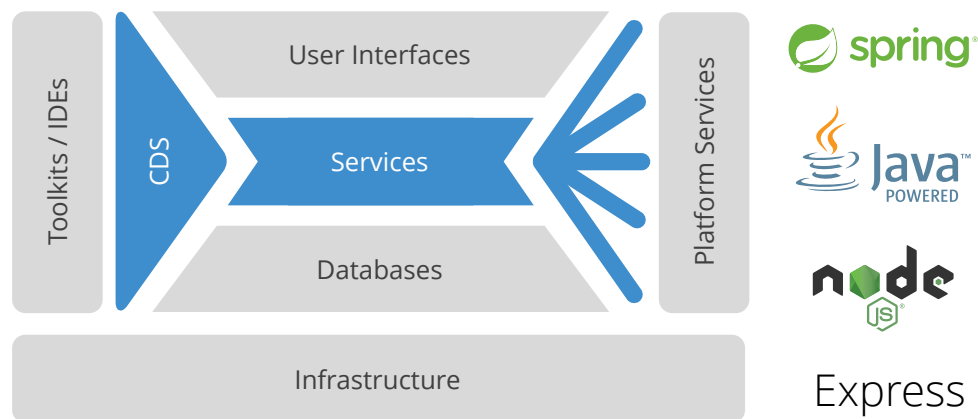


Abbildung 1 — CAP-Architektur mit CDS als zentralem Baustein [1]

Die technischen Features umfassen vollständige SDKs für die Laufzeitumgebungen Node.js und Java [13]. Event Handler werden als JavaScript-, TypeScript-Funktionen oder als Java-Klassen implementiert. CAP unterstützt mehrere Kommunikationsprotokolle wie OData [17], OpenAPI [18] und AsyncAPI, die automatisch aus CDS-Modellen generiert werden. Weitere zentrale Features sind Multi-Tenancy-Unterstützung zur Verwaltung mehrerer Mandanten [14, Ch. 5], [19], Datenbank-Abstraktion für HANA, SQLite und PostgreSQL sowie Event-Driven Messaging für asynchrone Kommunikation.

Multi-Tenancy in SaaS-Anwendungen erfordert Isolation auf mehreren Ebenen [19]. Während Tenants dieselbe Anwendungsinstanz nutzen, müssen ihre Daten und Performance-Charakteristika isoliert bleiben. CAP adressiert dies durch konfigurierbare Tenant-Isolation auf Datenbank-Ebene und Request-basierte Tenant-Erkennung.

2.1.1 Core Data Services (CDS)

CDS ist die Modellierungssprache von SAP CAP [2]. Entwickler definieren in CDS die Struktur ihrer Anwendung. Entities repräsentieren Datenmodelle, Services definieren API-Schnittstellen und Associations beschreiben Beziehungen zwischen Entities. Die textuelle Syntax heißt CDL und wird in Dateien mit der Endung `.cds` geschrieben [20, Sec. Language Preliminaries].

Das CDS-Toolkit kompiliert alle `.cds`-Dateien eines Projekts zu CSN, der kompilierten Repräsentation des Gesamtmodells [21, Sec. Anatomy]. Abbildung 2 zeigt diesen Kompilierungsprozess. CSN repräsentiert das Modell als Datenstruktur und enthält die vollständige semantische Information aller Definitionen.

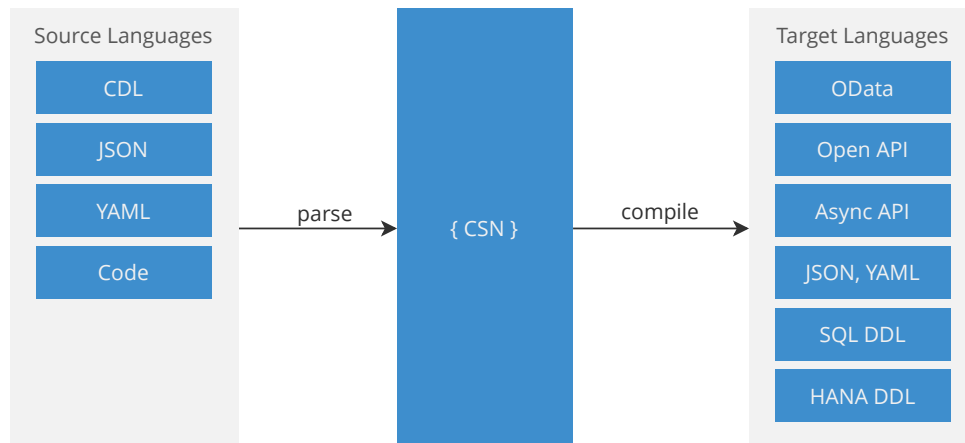


Abbildung 2 — CDS-Kompilierungsfluss mit CSN als zentralem Zwischenformat [2]

2.1.2 Core Schema Notation (CSN)

CSN ist die interne Repräsentation des CDS-Modells als JSON-Struktur [21, Sec. Anatomy]. Während CDL die textuelle Quellform ist, die Entwickler schreiben, ist CSN die vollständig aufgelöste und semantisch validierte Zielform, die Werkzeuge verarbeiten.

Das CSN-Format strukturiert alle Modelldefinitionen unter einem zentralen `definitions`-Objekt. Jede Definition ist über ihren vollqualifizierten Namen eindeutig identifizierbar [21, Sec. Structure]. Das Feld `kind` gibt den Typ der Definition an (z.B. `service`, `entity`, `type` oder `action`). Entities enthalten ein `elements`-Objekt, das alle Felder mit ihren Typen und Constraints beschreibt. Services sind als Container-Definitionen strukturiert, wobei die zu ihnen gehörenden Entities über den Namensraum zugeordnet werden.

CSN löst während der Kompilierung alle Referenzen, Imports und Vererbungen auf. Eine Entity, die einen Aspect einbindet, enthält in CSN alle geerbten Felder direkt [20, Sec. Aspects]. Ein Service, der Entities aus anderen Namespaces importiert, referenziert diese über vollqualifizierte Namen. Projections werden als

eigenständige Entities mit einem `projection`-Feld repräsentiert, das auf die Quell-Entity verweist [20, Sec. Projections]. Diese vollständige Auflösung macht CSN zu einer verlässlichen Quelle für semantische Analysen.

CSN dient als Ausgangsbasis für nachgelagerte Werkzeuge. Der CAP-Compiler generiert daraus Datenbankschemas, API-Dokumentationen und Laufzeit-Metadaten [21, Sec. Anatomy]. Extensions können CSN nutzen, um die Modellstruktur zu analysieren, ohne eigene Parser implementieren zu müssen. Allerdings enthält CSN standardmäßig keine Quellcode-Positionsinformationen, da es primär für die Codegenerierung gedacht ist, nicht für die Navigation im Quellcode.

Neben CDL und CSN umfasst CDS weitere Notationen wie Core Query Language (CQL) für Abfragen und Core Expression Language (CXL) für Ausdrücke [2]. Diese sind für die vorliegende Arbeit nicht relevant.

2.1.3 CDS-Artefakt-Typen

CDS definiert verschiedene Artefakt-Typen zur Modellierung von Anwendungen. Diese Artefakte werden in CDS-Dateien mit spezifischen Keywords deklariert und vom Language Server als Symbole mit entsprechenden Typ-Kennungen bereitgestellt. Tabelle 1 fasst die zentralen Artefakt-Typen mit ihrer Notation zusammen. Die Definitionen basieren auf der offiziellen CAP CDL-Dokumentation [20].

Artefakt-Typ	CDS-Keyword	Beschreibung
Entity	<code>entity</code>	Datenmodell mit Feldern, repräsentiert persistierte Datensätze
Type	<code>type</code>	Wiederverwendbare Typdefinition für Felder
Association	<code>association</code>	Deklarative Beziehung zwischen Entities
Aspect	<code>aspect</code>	Modellerweiterung zur Anreicherung von Entities
Service	<code>service</code>	Container für exponierte Entities und Operationen
Projection	<code>projection</code>	Abgeleitete Entity-Sicht mit Filterung
Action	<code>action</code>	Service-Operation mit Seiteneffekten
Function	<code>function</code>	Service-Operation ohne Seiteneffekte
Event Handler	–	Implementierung von Service-Logik (JS, TS oder Java)
Annotation	<code>@...</code>	Metadaten zur Anreicherung von Definitionen

Tabelle 1 — Zentrale CDS-Artefakt-Typen mit Notation und Verwendungskontext

In dieser Arbeit wird folgende Notationskonvention verwendet: Fachliche Bezüge auf Artefakt-Typen im Fließtext werden großgeschrieben (Entity, Service, Action). CDS-Keywords in Code-Beispielen werden in Monospace-Schrift kleingeschrieben (`entity`, `service`). Technische Bezeichner aus dem LSP-Kontext werden in Großbuchstaben geschrieben (ENTITY, SERVICE).

2.2 Language Server und Language Server Protocol

Moderne IDEs bieten intelligente Funktionen wie Autovervollständigung, Fehleranzeige, Go-to-Definition und Refactoring (z.B. Umbenennen von Symbolen). Diese Funktionen erfordern ein tiefes Verständnis der Programmiersprache, insbesondere von Syntax, Semantik und Typsystem. Ein Language Server ist ein separater Prozess, der diese Sprachanalyse übernimmt und die entsprechenden IDE-Features bereitstellt [22, Sec. What is LSP?], [23]. Die klare Trennung ermöglicht, dass der Server die „Language Smarts“ implementiert (Parsing, AST-Erzeugung, Analyse), während der Client sprachagnostisch bleibt und sich auf Editor-Interaktion konzentriert.

Das LSP ist ein von Microsoft entwickeltes Standardprotokoll zur Kommunikation zwischen IDE und Language Server [22, Sec. What is LSP?], [23]. Das Ziel ist die Entkopplung von Editor und Sprachanalyse. Ein Language Server kann dadurch in allen IDEs genutzt werden. Vor LSP musste jede IDE die Sprachunterstützung selbst implementieren, was zu $N \times M$ Implementierungen führte (N IDEs multipliziert mit M Sprachen). Mit LSP genügt es, den Language Server einmal zu implementieren, wodurch nur noch $N+M$ Implementierungen erforderlich sind [22, Sec. Why LSP?], [23].

2.2.1 Architektur des Language Servers

LSP basiert auf einem Client-Server-Modell. Die IDE ist der Client, der Language Server ist ein separater Prozess, der als Server fungiert [24, Sec. Base Protocol]. Der Server wird vom Client gestartet, verarbeitet während der Laufzeit Anfragen und wird am Ende wieder beendet. Der Server ist zustandsbehaftet und kennt den aktuellen Stand aller geöffneten Dokumente. Diese Architektur wurde durch

Erfahrungen mit Eclipse beeinflusst, wo Extensions im selben Prozess laufen und Startzeit oder Kernfunktionalität negativ beeinflussen konnten [23]. VS Code entschied sich deshalb für getrennte Extension-Prozesse und kontrollierte API-Gateways.

2.2.2 Kommunikationsprotokoll

Die Kommunikation zwischen IDE und Language Server basiert auf JSON-RPC 2.0 als Nachrichtenaustausch-Protokoll [25]. Das Protokoll ermöglicht Remote Procedure Calls, bei denen ein Client Methoden auf einem Server aufrufen kann, der in einem separaten Prozess läuft. Die Nachrichten werden als JSON-Objekte kodiert und über Standardein- und -ausgabe oder Netzwerkverbindungen übertragen.

Das Protokoll unterscheidet zwischen Requests, die eine Antwort erwarten, und Notifications, die asynchron ohne Antworterwartung gesendet werden. Diese Trennung ermöglicht sowohl synchrone Operationen wie „Hole die Definition eines Symbols“ als auch asynchrone Benachrichtigungen wie „Datei wurde geändert“. Die Verwendung von JSON als Nachrichtenformat macht das Protokoll sprachunabhängig und einfach implementierbar, da alle modernen Programmiersprachen JSON-Bibliotheken bereitstellen.

2.2.3 Relevante LSP-Funktionalitäten

LSP definiert verschiedene Funktionen für die Interaktion zwischen IDE und Language Server [24, Sec. Language Features]. Der Client sendet Requests an den Server, die jeweils eine Dokumentenposition oder einen Workspace-Kontext enthalten. Der Server analysiert den Quellcode und liefert die angeforderten Informationen als Response zurück. Diese Funktionen sind nicht auf klassische Programmiersprachen beschränkt, sondern können auch für domänenspezifische Sprachen implementiert werden [26]. Relevante Funktionen sind:

- `textDocument/completion`

Liefert Vervollständigungsvorschläge für die aktuelle Eingabeposition

- `textDocument/hover`

Liefert Typ- und Dokumentationsinformationen für ein Symbol an einer gegebenen Position

- `textDocument/definition`

Liefert die Quellcode-Position der Definition eines Symbols

- `workspace/symbol`

Liefert alle im Workspace definierten Symbole mit ihren Positionen

Neben diesen Kernfunktionen bieten Language Server in der Praxis oft zusätzliche Features wie Syntaxhervorhebung, kontextsensitive Autovervollständigung, Referenznavigation und gleichzeitiges Umbenennen aller Referenzen [26].

2.3 VS Code Extension Architektur

VS Code ist über Extensions erweiterbar [27]. Extensions sind Plug-ins, die zusätzliche Funktionalität zum Editor hinzufügen. Sie laufen in einem separaten Extension Host Prozess und sind damit vom Editor-Kern isoliert [28]. Extensions können UI-Elemente, Commands und Sprachfeatures beitragen.

VS Code stellt eine Extension API bereit, über die Extensions auf Editor-Funktionen zugreifen können [29]. Die API umfasst verschiedene Bereiche wie Dateiverwaltung, UI-Komponenten, Sprach-Features und Debugging-Funktionen. Eine dieser APIs ist die Tree View API.

2.3.1 UI-Extensions: Tree View und Webview API

VS Code bietet verschiedene APIs zur Implementierung eigener Benutzeroberflächen in Extensions. Die Wahl der passenden API hängt von der Komplexität und Art der darzustellenden Inhalte ab.

Die **Tree View API** ermöglicht das Erstellen hierarchischer Baumansichten in der Sidebar [30]. Das zentrale Konzept ist der `TreeDataProvider`, der die Methoden `getTreeItem()` und `getChildren()` implementiert [31]. Die erste Methode liefert die Darstellung eines Knotens mit Label, Icon und Zustand, während die zweite die

Kind-Elemente zurückgibt. VS Code ruft diese Methoden nur für aktuell sichtbare Knoten auf, wodurch auch große Baumstrukturen performant darstellbar sind. Die Aktualisierung erfolgt ereignisbasiert über ein `onDidChangeTreeData` Event [32].

Die **Webview API** ermöglicht die Darstellung von HTML-basiertem Inhalt mit vollständiger Kontrolle über HTML, CSS und JavaScript [33]. Die Kommunikation zwischen Extension und Webview erfolgt bidirektional über Message-Passing. Webviews bieten mehr Gestaltungsfreiheit als Tree Views, erfordern aber höheren Implementierungsaufwand.

2.3.2 Integration von Language Servern

VS Code bietet native Unterstützung für Language Server über die `vscode-languageclient` Bibliothek. Eine Extension kann einen Language Server als separaten Prozess starten. Der Language Client übernimmt die LSP-Kommunikation automatisch. Neben den Standard-LSP-Features kann eine Extension auch eigene Requests an den Server senden. So können Analyseergebnisse des Language Servers für eigene UI-Komponenten genutzt werden [34].

2.4 Navigation und Informationssuche in Softwaresystemen

Das Verständnis und die Wartung von Softwaresystemen erfordern kontinuierliche Navigation zwischen verschiedenen Artefakten. Entwickler verbringen einen erheblichen Teil ihrer Arbeitszeit damit, relevante Informationen zu lokalisieren, Abhängigkeiten nachzuvollziehen und Kontexte zu rekonstruieren. Die Art und Weise, wie Entwicklungsumgebungen Navigation unterstützen, hat direkten Einfluss auf die Produktivität und Kognitive Belastung von Entwicklern.

Beim Verstehen von Software müssen Entwickler mehrere Informationsebenen gleichzeitig verarbeiten. Die Cognitive Load Theory nach Sweller beschreibt, dass Menschen nur über eine begrenzte kognitive Verarbeitungskapazität im Arbeitsgedächtnis verfügen [35, p. 261]. Wenn diese Kapazität durch das Navigieren zwischen Dateien, das Rekonstruieren von Zusammenhängen oder das Merken verteilter Informationen erschöpft wird, bleibt weniger Kapazität für das eigentliche Problem. Kontextwechsel zwischen verschiedenen Dateien erhöhen die

kognitive Belastung zusätzlich, da Entwickler den vorherigen Kontext im Gedächtnis behalten müssen, während sie sich in einen neuen Kontext einarbeiten.

Ko et al. identifizieren in ihrer Studie zu Informationsbedürfnissen von Entwicklern, dass Entwickler häufig Schwierigkeiten haben, relevante Informationen über Code, Design und Programmausführung aus verschiedenen Artefakten zusammenzuführen [36, p. 347]. LaToza et al. zeigen, dass Entwickler stark auf implizites Code-Wissen angewiesen sind, das nicht in Werkzeugen oder Dokumentation externalisiert ist [37, p. 492]. Entwickler müssen mentale Modelle der Systemarchitektur aufbauen, die die verteilten Artefakte wieder zusammenführen. Dieser Prozess ist fehleranfällig und zeitaufwändig, insbesondere bei unbekannten oder großen Codebases.

Moderne IDEs bieten verschiedene Navigationsmuster an, die unterschiedliche Arten der Informationssuche unterstützen. File-basierte Navigation ermöglicht das Durchsuchen der Verzeichnisstruktur über einen Datei-Explorer. Symbol-basierte Navigation ermöglicht die direkte Suche nach Funktionen, Klassen oder Variablen, etwa durch Go-to-Definition [9, Sec. Go to Definition] oder Symbol-Search. Hierarchische Navigation zeigt Beziehungen zwischen Code-Elementen in Baumstrukturen, beispielsweise durch die Outline-Ansicht [10, Sec. Outline View]. Referenz-basierte Navigation ermöglicht das Auffinden aller Verwendungsstellen eines Symbols über Find References.

Diese existierenden Navigationsmuster operieren jedoch überwiegend datei- oder symbolzentriert. Eine servicezentrierte Navigation, die alle zu einer logischen Einheit gehörenden Artefakte aggregiert darstellt, ist in gängigen IDEs nicht vorhanden. Genau diese Lücke adressiert die vorliegende Arbeit für SAP CAP-Projekte.

3 Methodik

Diese Arbeit folgt dem DSR Ansatz zur Entwicklung und Evaluation eines technischen Artefakts. Das zu entwickelnde Artefakt ist eine VS Code-Extension, die Entwickler bei der Navigation in SAP CAP-Projekten unterstützen soll. Dieses Kapitel begründet zunächst die Wahl des methodischen Rahmens und beschreibt anschließend die konkrete Anwendung des DSR-Prozesses in dieser Arbeit.

3.1 Design Science Research als methodischer Rahmen

Die Wahl der Forschungsmethode hängt vom Forschungsziel ab. Verhaltenswissenschaftliche Ansätze erklären bestehende Phänomene und entwickeln Theorien. DSR hingegen konstruiert und evaluiert neue Artefakte zur Lösung identifizierter Probleme [12, p. 75]. Diese Arbeit entwickelt eine VS Code-Extension als konkretes technisches Artefakt, das Navigationsprobleme in CAP-Projekten löst. DSR ist daher die geeignete Forschungsmethode.

Alternative Forschungsansätze wurden geprüft und als ungeeignet befunden. Action Research ist ein Ansatz, bei dem Forscher aktiv in Veränderungsprozesse innerhalb einer Organisation eingreifen [38, p. 235]. Die Methode durchläuft iterative Zyklen von Diagnose, Planung, Aktion und Evaluation. Sie setzt voraus, dass Forscher und Praktiker eng in einem spezifischen organisationalen Kontext zusammenarbeiten. Für diese Arbeit ist Action Research ungeeignet, da keine organisationale Intervention angestrebt wird, sondern die Entwicklung eines generischen Werkzeugs.

Case Study Research ermöglicht die tiefgehende Analyse von Phänomenen in ihrem natürlichen Kontext. Case Studies sind explorativ, deskriptiv oder explanatorisch und zielen darauf ab, existierende Phänomene zu verstehen [39, p. 135]. Diese Arbeit hingegen entwickelt ein neues Artefakt und evaluiert dessen Eignung. Case Study Research ist daher nicht geeignet.

3.2 Eignung von DSR für diese Arbeit

Die Anwendung von DSR setzt voraus, dass das adressierte Problem bestimmte Charakteristika aufweist. Hevner et al. fordern, dass DSR technologiebasierte Lösungen für wichtige und relevante Probleme entwickelt [12, p. 82]. Das Problem muss zudem durch Konstruktion und Evaluation eines IT-Artefakts adressierbar sein [11, p. 49]. Diese Arbeit erfüllt diese Voraussetzungen aus folgenden Gründen.

Das Navigationsproblem in CAP-Projekten ist praxisrelevant und der aktuelle Entwicklungsprozess ineffizient. Wie Kapitel 4 zeigt, ist die physische Verteilung servicebezogener Artefakte über verschiedene Verzeichnisse mit einem hohen Navigationsaufwand verbunden. Die dort quantifizierten Szenarien belegen die Ineffizienz durch die hohe Anzahl erforderlicher Kontextwechsel zwischen Dateien.

VS Code bietet keine servicezentrierte Sicht auf CAP-Projekte. Existierende Navigationswerkzeuge folgen der technischen Ordnerstruktur. Eine aggregierte Darstellung aller zu einem Service gehörenden Artefakte existiert nicht. Das Problem ist somit durch Konstruktion eines Artefakts adressierbar.

Die Konstruktion einer servicezentrierten Navigationshilfe ist nicht trivial. Hevner et al. charakterisieren DSR-geeignete Probleme als „wicked problems“, die durch komplexe Interaktionen zwischen Problemkomponenten und Lösung gekennzeichnet sind [12, p. 81]. Die Gewinnung von Service-Strukturinformationen erfordert semantisches Verständnis der CAP-Architektur. Kapitel 5 vergleicht drei Lösungsansätze mit unterschiedlichen Trade-offs. Diese Notwendigkeit eines systematischen Vergleichs belegt die Nicht-Trivialität des Problems und die Forschungswürdigkeit der Arbeit.

3.3 Auswahl des DSR-Prozessmodells

Innerhalb von DSR existieren verschiedene Prozessmodelle, die den Forschungsprozess unterschiedlich strukturieren. Diese Modelle unterscheiden sich in Detaillierungsgrad, Phasenstruktur und Anwendbarkeit auf konkrete Forschungsprojekte.

Das konzeptionelle Framework von Hevner et al. bietet eine grundlegende Struktur mit drei Säulen (Environment, Information Systems Research, Knowledge Base) und den Phasen Build und Evaluate [12, pp. 79–80, Fig. 2]. Dieses Framework eignet sich zur grundsätzlichen Einordnung von DSR-Forschung, bietet jedoch keine detaillierte Prozessstruktur für die praktische Durchführung.

Dresch et al. definieren einen 12-schrittigen DSR-Prozess [40, pp. 118–124, Fig. 6.1], der sich in drei methodische Phasen gliedert. Das Modell unterscheidet explizit zwischen abduktiven, deduktiven und induktiven Schlussformen. Diese Fundierung ist zwar methodisch wertvoll, führt jedoch zu einem hohen Detaillierungsgrad, der für eine anwendungsorientierte Bachelorarbeit nicht erforderlich ist.

Peppers et al. schlagen ein 6-Phasen-Prozessmodell vor, das speziell für Information Systems Research entwickelt wurde. Das Modell strukturiert DSR-Forschung in die Phasen Problemidentifikation und Motivation, Zieldefinition, Entwurf und Entwicklung, Demonstration, Evaluation und Kommunikation [11, pp. 52–54]. Dieses Modell ist ein weitverbreiteter Standard in der Information-Systems-Community und bietet einen guten Kompromiss zwischen Strukturierung und Praktikabilität.

Für diese Arbeit wurde das Prozessmodell von Peppers et al. gewählt. Die Begründung liegt in mehreren Faktoren. Erstens ist Peppers' Prozessmodell ein weitverbreiteter DSR-Ansatz in der Information-Systems-Forschung, was die wissenschaftliche Anerkennung der Methodik stärkt. Zweitens ist das Modell explizit prozessgetrieben und flexibel: Peppers et al. beschreiben verschiedene Einstiegs-punkte in den Prozess (problem-centered, objective-centered, design-centered, client-context-initiated) [11, p. 56], was die Anwendbarkeit auf unterschiedliche Forschungskontexte erhöht. Drittens bietet die 6-Phasen-Struktur einen klaren Rahmen, der im Vergleich zu Dresch's 12 Schritten keine unnötige Komplexität einführt. Viertens enthält das Modell eine explizite Demonstration-Phase, die ideal zur prototypischen Implementierung dieser Arbeit passt, während Hevner's Framework diese Phase nicht explizit ausweist.

3.4 Anwendung des DSR-Prozesses in dieser Arbeit

Die sechs Phasen des DSR-Prozesses nach Peffers et al. [11, pp. 52–56] strukturieren den Forschungsprozess dieser Arbeit. Die Phasen dienen als methodischer Rahmen, während die Kapitelstruktur der fachlichen Logik des Untersuchungsgegenstands folgt.

Phase 1: Problemidentifikation und Motivation

Kapitel 4 identifiziert das Navigationsproblem in CAP-Projekten durch Analyse typischer Repository-Strukturen und quantifiziert den Navigationsaufwand anhand repräsentativer Entwicklungsszenarien.

Phase 2: Zieldefinition

Ebenfalls in Kapitel 4 werden aus den identifizierten Herausforderungen lösungsneutrale Designziele abgeleitet und in konkrete funktionale sowie nicht-funktionale Anforderungen überführt.

Phase 3: Entwurf und Entwicklung

Kapitel 5 vergleicht drei Integrationsstrategien zur Gewinnung von Service-Strukturinformationen mittels Nutzwertanalyse und wählt die vorteilhafteste aus. Kapitel 6 implementiert die gewählte Strategie als VS Code-Extension.

Phase 4: Demonstration

Kapitel 6 demonstriert die praktische Anwendbarkeit der Extension am Bookshop-Referenzprojekt.

Phase 5: Evaluation

Kapitel 7 evaluiert das Artefakt kriterienbasiert anhand der definierten Anforderungen durch praktische Erprobung an repräsentativen CAP-Projekten.

Phase 6: Kommunikation

Kapitel 8 interpretiert die Ergebnisse, diskutiert Limitationen, fasst die Erkenntnisse zusammen und reflektiert den Beitrag zur Wissensbasis. Diese Arbeit selbst stellt die primäre Kommunikationsform dar.

4 Problem- und Anforderungsanalyse

Dieses Kapitel bildet die ersten beiden Phasen des DSR-Prozesses nach Peffers et al. [11]. Die erste Phase (Problemidentifikation und Motivation) identifiziert und konkretisiert das Navigationsproblem in SAP CAP-Projekten. Die zweite Phase (Zieldefinition) definiert Designziele und Anforderungen an die zu entwickelnde VS Code-Extension. Zunächst wird die strukturelle Organisation typischer CAP-Repositories analysiert und die Verteilung servicebezogener Artefakte untersucht. Darauf aufbauend werden navigationsbezogene Herausforderungen abgeleitet, aus denen Designziele sowie funktionale und nicht-funktionale Anforderungen formuliert werden. Abschließend wird die Arbeit inhaltlich abgegrenzt.

4.1 Problemkontext und Analyse von CAP-Repositories

Die folgenden Abschnitte analysieren die strukturellen Charakteristika von CAP-Projekten und identifizieren daraus resultierende Navigationsherausforderungen. Zunächst wird die standardisierte Projektstruktur untersucht, anschließend die Verteilung servicebezogener Artefakte analysiert.

4.1.1 Typische Struktur von CAP-Projekten

SAP CAP gibt eine strenge Projektstruktur vor, die das Prinzip der Separation of Concerns umsetzt und damit die Wartbarkeit skalierender Projekte unterstützt [7, Sec. Separation of Concerns]. Ein typisches CAP-Projekt folgt der in Abbildung 1 dargestellten Struktur.

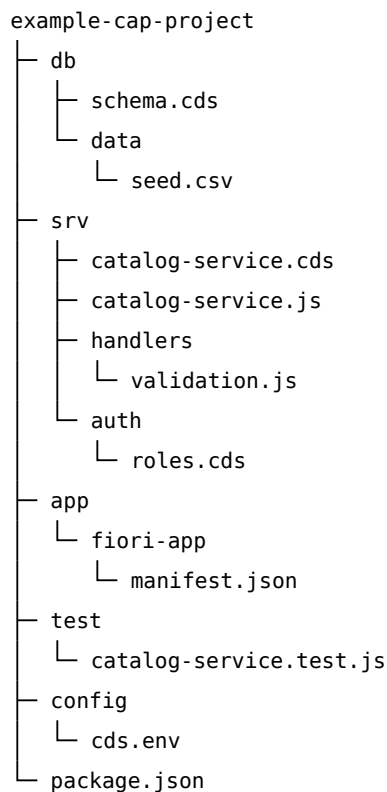


Abbildung 1 — Exemplarische Struktur eines SAP CAP-Projekts (vereinfacht) [6].

example-cap-project/

Das Rootverzeichnis bildet die Basis des Projekts und bündelt alle fachlichen, technischen und konfigurativen Artefakte nach dem Prinzip der Separation of Concerns.

db/

Dieses Verzeichnis enthält das fachliche Domänenmodell. Hier werden Entities und ihre Beziehungen in CDS definiert, die die Persistenz- und Datenstrukturebene repräsentieren.

srv/

Dieses Verzeichnis beinhaltet Service-Definitionen und Geschäftslogik. Der Service exponiert ausgewählte Teile des Domänenmodells und verknüpft deklarative Modellierung mit imperativer Implementierung.

app/

Dieses Verzeichnis enthält optionale Frontend-Anwendungen, die die definierten Services konsumieren und die Präsentationsebene repräsentieren.

test/

Dieses Verzeichnis beinhaltet Testfälle zur Validierung der Service-Logik und unterstützt damit Qualitätssicherung und Wartbarkeit.

config/

Dieses Verzeichnis enthält umgebungsspezifische Konfigurationen zur Steuerung von Laufzeit- und Deployment-Parametern.

package.json

Diese Datei definiert Projektmetadaten, Abhängigkeiten sowie Skripte für Build, Start und Tests.

4.1.2 Verteilung servicebezogener Artefakte

Ein CAP-Service besteht nicht aus einem einzelnen Artefakt, sondern setzt sich aus mehreren funktionalen Bereichen zusammen, die unterschiedlichen Verantwortlichkeiten zugeordnet sind. Diese Fragmentierung erfolgt entlang technischer und fachlicher Concerns, wobei die Zuordnung zwischen den Artefakten über das Projekt verteilt ist.

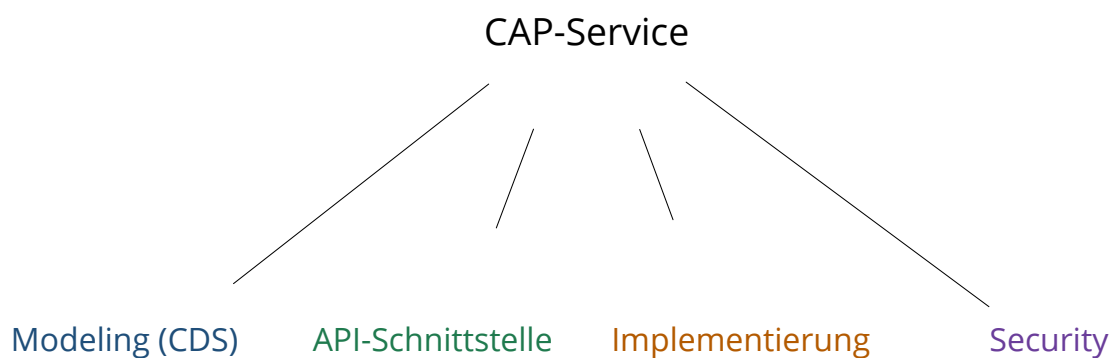


Abbildung 3 — Konzeptionelle Fragmentierung eines CAP-Services (vereinfacht).

Abbildung 3 verdeutlicht, dass ein CAP-Service nicht als monolithisches Artefakt vorliegt, sondern sich über mehrere konzeptionelle Ebenen erstreckt. Modell- und Service-Definition sind konzeptionell getrennt, wobei die Service-Schnittstelle nicht zwingend das vollständige Domänenmodell abbildet. Das Laufzeitverhalten ist nicht im Modell selbst definiert, sondern in separaten Implementierungen ausgelagert. Sicherheitsrelevante Regeln werden als Annotationen außerhalb der Kernlogik verortet. Dadurch sind fachliche und technische Aspekte strukturell entkoppelt [41, Sec. Focus on Domain].

Typ	Ebene	Beispiele
Entity	Modeling (CDS)	Books, Authors
Type	Modeling (CDS)	UUID, String, Decimal
Association	Modeling (CDS)	to Author, to Items
Aspect	Modeling (CDS)	Reusable model extensions
Service	API-Schnittstelle	CatalogService
Projection	API-Schnittstelle	Books as projection on ...
Action / Function	API-Schnittstelle	submitOrder(), topSelling()
Event Handler	Implementierung	before / on / after
Authorization Annotations	Security	@restrict, @requires

Tabelle 2 — Zentrale Artefakte eines CAP-Services (basierend auf [4, Sec. Domain Modeling] Authorization Annotations: [5, Sec. Role-Based Access Control])

Tabelle 2 konkretisiert die in der Abbildung 3 dargestellten Ebenen durch die zentralen Artefakt-Typen des CAP-Modells. Ein vollständiges Serviceverständnis erfordert die Betrachtung mehrerer Artefakt-Kategorien, deren Beziehungen über Referenzen hergestellt werden. Eine zentrale, aggregierte Repräsentation eines Services existiert nicht auf Artefaktebene. Änderungen an einem Service können daher mehrere Ebenen gleichzeitig betreffen, da sich die Service-Identität erst aus dem Zusammenspiel aller Ebenen ergibt. Diese Architektur folgt den Prinzipien von Domain-Driven Design [42, Sec. 1.1] und Separation of Concerns [4, Sec. Separation of Concerns], [41, Sec. Focus on Domain].

4.2 Identifizierte Herausforderungen bei der Navigation

Aus der analysierten Projektstruktur und Artefaktverteilung ergeben sich mehrere Herausforderungen für die Navigation in CAP-Projekten. Die folgenden Abschnitte beschreiben diese Herausforderungen und quantifizieren deren Auswirkungen anhand repräsentativer Entwicklungsszenarien.

4.2.1 Verteilte Artefakte und implizite Abhängigkeiten

Ein CAP-Service erstreckt sich über mehrere konzeptionelle Ebenen, wobei Modell, Schnittstelle, Implementierung und Sicherheitsregeln physisch getrennt sind. Die Beziehungen zwischen diesen Artefakten sind nicht visuell auf einen Blick erkenn-

bar. Projections referenzieren Entities, Event Handlers referenzieren Service-Operationen, und Sicherheitsannotationen beeinflussen das Verhalten ohne zentrale Sichtbarkeit. Kein einzelnes Artefakt enthält die vollständige Service-Definition.

Diese Problematik entspricht dem von Tarr et al. beschriebenen Phänomen der „Tyrannei der dominanten Dekomposition“ [43, p. 809]: Software wird typischerweise entlang einer primären Dimension strukturiert. Im Fall von CAP-Projekten nach Ordern und Dateien entsprechend technischer Verantwortlichkeiten. Andere wichtige Dimensionen, wie die servicezentrierte fachliche Sicht, werden dadurch über das Projekt verteilt und sind nicht mehr unmittelbar sichtbar [4].

4.2.2 Erhöhter Navigationsaufwand und fehlende Service-Übersicht

Mit zunehmender Weiterentwicklung eines Softwaresystems steigt dessen strukturelle Komplexität. Lehman beschreibt dieses Phänomen im Gesetz der zunehmenden Komplexität, wonach die Komplexität eines Systems mit fortlaufenden Änderungen wächst, sofern keine Maßnahmen zur Reduktion dieser Komplexität ergriffen werden [44, p. 1068]. Zur Analyse eines Services sind mehrere Dateiwchsel erforderlich, etwa zwischen Datenbankmodell, Implementierung und Sicherheitsregeln. Da keine aggregierte Sicht auf alle servicebezogenen Artefakte existiert, muss die Struktur eines Services rekonstruiert werden. Diese Kontextwechsel erhöhen die Kognitive Belastung, da Menschen nur über eine begrenzte kognitive Verarbeitungskapazität verfügen und mehrere Informationen gleichzeitig berücksichtigen müssen [35, p. 261].

4.2.3 Quantifizierung anhand konkreter Szenarien

Die beschriebenen Navigationsprobleme lassen sich anhand typischer Entwicklungsaufgaben im Bookshop-Beispielprojekt [6] konkretisieren. Vier repräsentative Szenarien wurden durch manuelle Analyse untersucht, um den erforderlichen Navigationsaufwand zu quantifizieren.

Das Szenario mit dem höchsten Navigationsaufwand ist „Service verstehen“. Es erfordert zehn Navigationsschritte über drei Dateien und zeigt die Herausforderung exemplarisch. Ein Entwickler soll sich einen Überblick über den CatalogService

verschaffen und dessen Struktur nachvollziehen. Tabelle 3 dokumentiert die erforderlichen Navigationsschritte. Der Entwickler muss zunächst in der Service-Definition Import-Statement, Namespace, Projections und Actions analysieren. Da Projections auf das Datenmodell verweisen, ist ein Wechsel zur Datenbank-Schema-Datei erforderlich, wo die Entity Books sowie die referenzierten Entities Authors und Genres verstanden werden müssen. Abschließend muss die Handler-Implementierung mit Event Handlers analysiert werden. Dieser Workflow erfordert insgesamt drei Dateien und zehn separate Navigationsschritte zwischen verschiedenen Artefakten.

#	Datei	Zeile	Aktion
1	srv/cat-service.cds	1	Import-Statement und Namespace identifizieren
2	srv/cat-service.cds	3	Service-Definition CatalogService analysieren
3	srv/cat-service.cds	6–9	Projection ListOfBooks verstehen
4	srv/cat-service.cds	12–16	Projection Books verstehen
5	srv/cat-service.cds	19	Action submitOrder identifizieren
6	db/schema.cds	4–13	Entity Books mit Associations verstehen
7	db/schema.cds	15–23	Referenzierte Entity Authors analysieren
8	db/schema.cds	26–29	Referenzierte Entity Genres analysieren
9	srv/cat-service.js	3–6	Handler-Klasse und Entity-Referenzen
10	srv/cat-service.js	9–23	Event Handlers (after, on) analysieren

Tabelle 3 — Szenario „Service verstehen“ im Bookshop-Projekt

Die weiteren Szenarien „Feld hinzufügen“, „Auth anpassen“ und „Handler debuggen“ zeigen vergleichbare Navigationsmuster und sind im Anhang detailliert dokumentiert. Abbildung 4 fasst die Navigationskomplexität aller vier Szenarien zusammen.

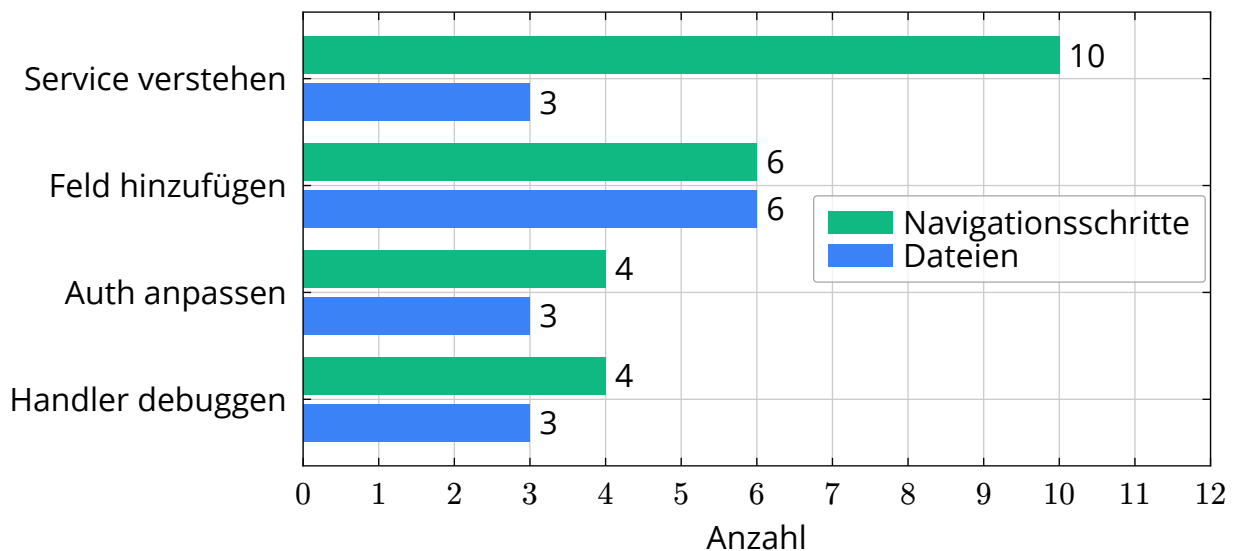


Abbildung 4 — Navigationskomplexität typischer Entwicklungsaufgaben im Bookshop-Projekt

4.2.4 Bestehende Ansätze zur Navigation in Entwicklungsumgebungen

Moderne Entwicklungsumgebungen bieten verschiedene Navigationsmechanismen. VS Code stellt grundlegende Funktionen wie die Outline-Ansicht [10, Sec. Outline View] und Go-to-Definition [9, Sec. Go to Definition] bereit. Die SAP CDS Language Support Extension [45] erweitert dies um CDS-spezifische Features wie Syntaxhervorhebung und Autovervollständigung.

Trotz dieser Funktionen zeigt die Forschung grundlegende Probleme. Ko et al. identifizieren, dass Entwickler häufig Schwierigkeiten haben, relevante Informationen über Code, Design und Programmausführung aus verschiedenen Artefakten zusammenzuführen [36, p. 347]. LaToza et al. zeigen zudem, dass Entwickler stark auf implizites Code-Wissen angewiesen sind, welches nicht in Werkzeugen oder Dokumentation externalisiert ist [37, p. 492].

Die zentrale Limitation besteht darin, dass diese Werkzeuge überwiegend datei- und symbolzentriert operieren. Eine servicezentrierte Aggregation über mehrere Dateien und Ebenen hinweg wird nicht unterstützt.

4.3 Ableitung der Designziele

Aus den identifizierten Navigationsproblemen werden nun zentrale Designziele abgeleitet. Diese beschreiben lösungsneutral, welche Eigenschaften eine geeignete Lösung aufweisen muss. Die Designziele bilden die Grundlage für die anschließende Definition funktionaler und nicht-funktionaler Anforderungen.

Aggregierte Service-Sicht

Alle Artefakte eines Services müssen gebündelt dargestellt werden. Die Lösung muss eine einheitliche Einstiegsperspektive auf einen Service bieten, in der Modell, Schnittstelle, Implementierung und Sicherheitsaspekte gemeinsam sichtbar sind. Die Service-Identität muss ohne manuelle Rekonstruktion erfassbar sein.

Reduktion von Navigations- und Kontextwechseln

Die Anzahl notwendiger Dateiwechsel muss minimiert werden. Deklarative und imperative Artefakte müssen kontextnah zugänglich sein. Relevante Artefakte müssen schnell auffindbar sein. Kognitive Kontextwechsel zwischen den Ebenen müssen reduziert werden.

Explizite Darstellung von Abhängigkeiten

Beziehungen zwischen allen Arten von Entitäten müssen sichtbar gemacht werden. Verknüpfungen zwischen Service-Definition und Implementierung müssen nachvollziehbar sein. Sicherheitsrelevante Annotationen müssen eindeutig zuordbar sein. Referenzen müssen transparent werden.

Servicezentrierte Navigation

Die Navigation muss sich an den fachlichen Services orientieren. Die technische Ordnerstruktur darf nicht primärer Navigationsanker sein. Der Service muss als logische Einheit im Mittelpunkt stehen.

4.4 Funktionale Anforderungen

Aus den zuvor formulierten Designzielen lassen sich konkrete funktionale Anforderungen an die zu entwickelnde VS Code-Extension ableiten. Während die Designziele die gewünschten Eigenschaften der Lösung auf konzeptioneller Ebene

beschreiben, spezifizieren die folgenden Anforderungen die Funktionen, die das Artefakt zur Unterstützung der Navigation in CAP-Projekten bereitstellen muss.

FA1 Identifikation von CAP-Services

Die Extension muss CAP-Services innerhalb eines Projekts automatisch identifizieren und alle Services anzeigen.

FA2 Aggregierte Serviceübersicht

Die Extension muss eine aggregierte Darstellung aller zu einem Service gehörenden Artefakte bereitstellen.

FA3 Navigation zu Artefakten

Die Extension muss eine direkte Navigation von der Serviceübersicht zu den entsprechenden Artefakten im Quellcode ermöglichen.

FA4 Darstellung von Artefaktbeziehungen

Die Extension muss Beziehungen und Hierarchien zwischen servicebezogenen Artefakten sichtbar machen.

4.5 Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen ergeben sich aus dem Nutzungskontext innerhalb der Entwicklungsumgebung zusätzliche nicht-funktionale Anforderungen. Diese beschreiben qualitative Eigenschaften und Rahmenbedingungen, die bei der Umsetzung der Extension berücksichtigt werden müssen.

NFA1 Integration in Visual Studio Code

Die Lösung muss als Extension für die Entwicklungsumgebung VS Code implementiert werden und sich in deren Erweiterungsmechanismus [27] integrieren.

NFA2 Kompatibilität mit CAP-Projektstrukturen

Die Extension muss mit der typischen Struktur von SAP CAP-Projekten kompatibel sein und relevante Artefakte in den vorgesehenen Verzeichnissen erkennen können.

NFA3 Performante Analyse von Projekten

Die Extension muss CAP-Projekte effizient analysieren. Das initiale Laden des Modells beim Projektöffnen muss innerhalb von 10 Sekunden abgeschlossen sein, um die Aufmerksamkeit des Entwicklers zu halten. Alle nachfolgenden Navigati-

onsoperationen müssen in unter 1 Sekunde erfolgen, um den Gedankenfluss nicht zu unterbrechen [46, p. 135].

NFA4 Aktualität der Artefaktübersicht

Änderungen an relevanten Projektartefakten müssen automatisch erkannt werden, sodass die dargestellte Serviceübersicht den aktuellen Projektzustand widerspiegelt.

4.6 Abgrenzung der Arbeit

Die vorliegende Arbeit fokussiert auf die Unterstützung der Navigation in CAP-Projekten durch eine VS Code-Extension. Daraus ergeben sich bewusste Einschränkungen des Untersuchungs- und Lösungsumfangs.

Ziel der Extension ist die Verbesserung der Navigation zwischen servicebezogenen Artefakten. Funktionen zur automatischen Codegenerierung oder Refaktorisierung sind explizit nicht Bestandteil der Lösung.

Die Evaluation erfolgt anhand ausgewählter CAP-Projekte. Eine umfassende Nutzerstudie mit CAP-Entwicklern ist nicht Bestandteil der Arbeit.

5 Integrationsstrategien

Dieses Kapitel bildet die dritte Phase des DSR-Prozesses nach Peffers et al. [11] (Entwurf und Entwicklung). Es entwickelt und vergleicht drei Integrationsstrategien für die Gewinnung von Service-Strukturinformationen aus CAP-Projekten. Die Strategien unterscheiden sich in ihrer Datenquelle und dem Integrationsansatz. Repository-basiert durch direktes Parsen, CSN-basiert durch Nutzung des kompilierten Modells und Language Server-basiert durch Integration des CDS Language Servers. Jede Strategie wird hinsichtlich Implementierungsaufwand, semantischer Genauigkeit, Verfügbarkeit von Positionsinformationen, Integrationsgrad, Wartbarkeit und Abhängigkeiten bewertet. Der Vergleich mündet in die Auswahl einer Strategie, die als Grundlage für die Prototyp-Implementierung in Kapitel 6 dient.

5.1 Repository-basierte Integration

Die repository-basierte Integration rekonstruiert die Service-Struktur direkt aus den Dateien des Projektrepositories. Die Analyse erfolgt durch Parsing der `.cds`-Dateien, wobei Entities, Services, Projections und Operationen syntaktisch extrahiert werden. Die Implementierungslogik wird durch Analyse von `.js`- und `.ts`-Dateien ermittelt. Beziehungen zwischen Artefakten müssen aus Referenzen im Code rekonstruiert werden.

5.1.1 Struktur von CDS-Dateien

Quellcode 2 zeigt einen Ausschnitt einer typischen CDS-Service-Definition, anhand derer die Parsing-Herausforderungen verdeutlicht werden können.

```
using { sap.capire.bookshop as my } from '../db/schema'

service CatalogService {
  @readonly entity Books as projection on my.Books {
    *, author.name as author, genre.name as genre
  } excluding { createdBy, modifiedBy };

  @requires: ',authenticated-user'
  action submitOrder ( book: Books:ID, quantity: Integer );
}
```

Quellcode 2 — CDS-Service-Definition mit Projections und Action [6]

5.1.2 Herausforderungen beim Parsing von CDS

Die technische Umsetzung erfordert einen eigenen Parser für die CDS-Sprache. CDS ist eine Domänenspezifische Sprache mit eigener Grammatik, die Konstrukte wie `entity`, `service`, `projection`, `aspect` und `type` unterstützt [20]. Die Implementierung eines Parsers durchläuft typischerweise mehrere Phasen: lexikalische Analyse (Tokenisierung), syntaktische Analyse (Aufbau eines Parse-Baums) und semantische Analyse (Typprüfung, Referenzauflösung) [47, Ch. 1, pp. 4–5].

Bereits Quellcode 2 zeigt mehrere Parsing-Herausforderungen. Der `using`-Import in Zeile 1 führt einen Namespace-Alias (`my`) ein, der im gesamten Service verwendet wird. Ein Parser muss diese Alias-Zuordnung in einer Symboltabelle verwalten und bei jeder Referenz auflösen können. Die Projection `Books` in Zeile 4 referenziert `my.Books`, wobei `my` auf `sap.capire.bookshop` aufgelöst werden muss. Zusätzlich enthält die Projection Feld-Aliasing (`author.name as author`), wobei Pfad-Ausdrücke über Associations aufgelöst werden müssen.

Die Grammatik von CDS unterstützt zudem Annotationen (`@readonly`, `@requires`), die syntaktisch an verschiedenen Stellen auftreten können. Annotationen können

verschachtelt sein und komplex strukturierte Werte enthalten [20]. Ein vollständiger Parser muss diese Annotationen extrahieren und den entsprechenden Artefakten zuordnen können. Actions wie `submitOrder` definieren Parametertypen (`Books: ID, Integer`), wobei die Typen ebenfalls im Modell aufgelöst werden müssen.

5.1.3 Referenzauflösung über Dateigrenzen

Eine zentrale Herausforderung der repository-basierten Integration ist die Referenzauflösung über Dateigrenzen hinweg. CDS-Dateien können sich gegenseitig importieren, wobei Imports relative Pfade (`'../db/schema'`) oder Namespace-Referenzen verwenden. Ein Parser muss diese Imports auflösen, die referenzierten Dateien laden und deren Definitionen in eine globale Symboltabelle integrieren.

Zusätzlich müssen Event Handler in `.js`- oder `.ts`-Dateien den entsprechenden Services zugeordnet werden. Die Zuordnung erfolgt typischerweise durch Namenskonventionen (z.B. `srv/catalog-service.cds` und `srv/catalog-service.js`) oder durch explizite Registrierung im Handler-Code. Ein vollständiger Ansatz müsste auch JavaScript-Dateien analysieren, um diese Zuordnungen zu erkennen. Dies erfordert zusätzlich einen JavaScript-Parser oder die Nutzung spezialisierter Parsing-Bibliotheken.

5.1.4 Implementierungsaufwand und semantische Grenzen

Dieser Ansatz bringt einen hohen Implementierungsaufwand für Parsing und Referenzauflösung mit sich. Die Implementierung eines vollständigen CDS-Parsers mit korrekter Symboltabellenverwaltung und Multi-File-Referenzauflösung würde mehrere Personenmonate in Anspruch nehmen. Zudem besteht das Risiko semantischer Fehlinterpretationen der CDS-Sprache, da ein eigener Parser die Sprachsemantik nicht vollständig abbilden kann. Sprachfeatures wie Aspects, Vererbung und berechnete Elemente erfordern semantische Analysen, die über syntaktisches Parsing hinausgehen [47, Ch. 6, pp. 373–376].

Darüber hinaus dupliziert dieser Ansatz Analysefunktionen, die bereits durch das CAP-Tooling bereitgestellt werden. Der CAP-Compiler führt bereits eine vollständige Analyse der CDS-Dateien durch, einschließlich Typprüfung, Referenzauflösung

und Validierung. Ein eigener Parser würde diese Logik erneut implementieren müssen, was sowohl redundant als auch fehleranfällig ist. Bei Änderungen an der CDS-Sprache müsste der Parser manuell aktualisiert werden, was die Wartbarkeit beeinträchtigt.

Der Vorteil dieses Ansatzes liegt in seiner hohen Flexibilität und Unabhängigkeit. Ein eigener Parser ermöglicht die vollständige Kontrolle über die extrahierten Informationen und ist unabhängig von externen Tools. Der Ansatz könnte auch ohne installierten CAP-Compiler funktionieren, was in bestimmten Entwicklungsumgebungen relevant sein kann. Zudem wären auch Analysefunktionen möglich, die über die vom CAP-Tooling bereitgestellten Informationen hinausgehen, etwa statische Code-Analyse oder projektspezifische Konventionsprüfungen.

5.2 CSN-basierte Integration

Die CSN-basierte Integration nutzt das von CAP generierte CSN Modell als Informationsquelle. CSN ist eine kompilierte JSON-Repräsentation aller CDS-Artefakte. Der CAP-Compiler erzeugt CSN aus den .cds-Quelldateien und löst dabei alle Referenzen, Imports und Vererbungen auf.

5.2.1 Struktur und Inhalt von CSN

Im CSN-Modell sind Entities, Services, Projections und Operationen strukturiert enthalten. Jedes Artefakt ist unter seinem vollqualifizierten Namen als JSON-Objekt abgelegt. Das Feld `kind` gibt dabei den Artefakttyp an (z.B. "entity", "service", "function"). Beziehungen zwischen Artefakten sind bereits explizit aufgelöst, so dass keine eigene Referenzauflösung erforderlich ist.

Die hierarchische Struktur von CSN spiegelt die semantische Struktur des CDS-Modells wider. Services werden als Top-Level-Definitionen mit `kind: "service"` repräsentiert. Entities innerhalb eines Services erhalten einen vollqualifizierten Namen wie `ServiceName.EntityName`. Das `projection`-Feld enthält Informationen über die Quell-Entity, auf die die Projection verweist. Das `elements`-Feld listet alle Felder der Entity mit ihren Typen und Eigenschaften auf.

Quellcode 3 zeigt einen Ausschnitt eines CSN-Modells mit einem Service und einer zugehörigen Entity.

```
{
  „definitions“: {
    „TravelService“: {
      „kind“: „service“,
      „$location“: { „file“: „srv/travel-service.cds“, „line“: 5 }
    },
    „TravelService.Travels“: {
      „kind“: „entity“,
      „projection“: {
        „from“: { „ref“: [„sap.capire.travels.Travels“] }
      },
      „elements“: {
        „ID“: { „key“: true, „type“: „cds.Integer“ },
        „Description“: { „type“: „cds.String“, „length“: 1024 },
        „BeginDate“: { „type“: „cds.Date“ }
      }
    }
  }
}
```

Quellcode 3 — Ausschnitt eines CSN-Modells mit Service und Entity

Aus Quellcode 3 wird die Struktur ersichtlich. Der Service `TravelService` ist als Top-Level-Definition mit `kind: "service"` und einem `$location`-Feld vorhanden. Die Entity `TravelService.Travels` ist als Projection definiert, wobei das `projection.from.ref`-Feld auf die Quell-Entity `sap.capire.travels.Travels` verweist. Die `elements` enthalten die Felder der Entity mit vollständigen Typ-Informationen. Felder mit der Eigenschaft `key: true` kennzeichnen Primärschlüssel.

5.2.2 CSN-Generierung und Build-Integration

CSN wird durch den CAP-Compiler erzeugt. Der Befehl `cds compile` liest alle `.cds`-Dateien im Projekt, führt die semantische Analyse durch und schreibt das Ergebnis als JSON in eine Datei. Die Generierung erfolgt typischerweise während des Build-Prozesses oder on-demand durch die Extension.

Für die Nutzung in einer VS Code-Extension könnte CSN auf zwei Arten bereitgestellt werden. Erstens durch direkten Aufruf des CAP-Compilers aus der Extension heraus mittels eines Child-Prozesses. Dies würde bei jedem Projektöffnen oder jeder Dateiänderung eine Neu-Kompilierung erfordern. Zweitens durch Nutzung einer bereits generierten CSN-Datei, falls diese im Build-Prozess erzeugt wird. Dies setzt jedoch voraus, dass Entwickler den Build-Schritt ausgeführt haben, was in frühen Entwicklungsphasen nicht immer der Fall ist.

Die Performance der CSN-Generierung ist für große Projekte relevant. Der CAP-Compiler muss alle CDS-Dateien parsen, alle Referenzen auflösen und das vollständige Modell aufbauen. Bei Projekten mit mehreren hundert CDS-Dateien kann dies mehrere Sekunden dauern. Für eine responsive Extension wäre eine Caching-Strategie erforderlich, die nur bei tatsächlichen Änderungen eine Neu-Kompilierung anstößt.

5.2.3 Limitationen der location-Daten

CSN bietet optional Positionsinformationen über das Feld `$location`, die durch den Kompilierungsmodus `--locations` aktiviert werden können. Wie in Quellcode 3 ersichtlich, enthält `$location` die Datei (file) und Zeile (line) der Definition. Diese Informationen sind jedoch unvollständig und weisen mehrere Limitationen auf.

Nicht alle Artefakte erhalten `$location`-Daten. Top-Level-Definitionen wie Services und Entities besitzen typischerweise Location-Informationen. Verschachtelte Elemente wie einzelne Felder oder Annotations erhalten jedoch oft keine Positionsdaten. Actions und Functions innerhalb eines Service können Location-Daten haben, aber Parameter und Rückgabewerte sind nicht immer präzise lokalisiert. Diese Unvollständigkeit macht CSN für FA3 (direkte Navigation zu Artefakten) nur eingeschränkt nutzbar.

Zusätzlich fehlen Spalten-Informationen (character-Offset). CSN liefert nur die Zeile, nicht jedoch die genaue Position innerhalb der Zeile. Für präzise Navigation zu kleinen Artefakten wie einzelnen Feldern oder Parametern ist dies unzureichend. Die Extension müsste zusätzliche Heuristiken implementieren, um die genaue

Position innerhalb einer Zeile zu bestimmen, etwa durch Textsuche nach dem Artefaktnamen.

5.2.4 Vorteile und Einschränkungen

Die semantisch korrekte Interpretation ist durch die Nutzung des CAP-Compilers garantiert. Das JSON-Format ist direkt parsebar und erfordert keinen eigenen CDS-Parser. Der Implementierungsaufwand für das Parsen von CSN ist gering, da Standard-JSON-Bibliotheken verwendet werden können. Die hierarchische Struktur mit vollqualifizierten Namen erleichtert die Zuordnung von Artefakten zu Services.

Die Haupteinschränkung liegt in den unvollständigen Positionsinformationen. Für FA3 (direkte Navigation zu Artefakten) ist diese Abdeckung nicht ausreichend. Die Extension müsste Fallback-Mechanismen implementieren, etwa Textsuche oder Heuristiken, um fehlende Positionen zu approximieren. Zusätzlich ist CSN integraler Bestandteil des CAP-Compilers. Erweiterungen am CSN-Format würden den Build-Prozess verlangsamen und sind daher nicht praktikabel. Die Extension wäre auf die vom CAP-Compiler bereitgestellten Informationen beschränkt.

5.3 Language-Server-basierte Integration

Die Language Server-basierte Integration nutzt den CAP Language Server als zentrale Analysequelle (siehe Kapitel 2). Der Language Server stellt semantische Analysen der CDS-Artefakte bereit und kommuniziert über das LSP mit VS Code.

5.3.1 CAP Language Server Architektur

Der CAP Language Server ist als eigenständiger Prozess implementiert, der parallel zur VS Code-Instanz läuft. Die Kommunikation erfolgt über Standard-Input/Output oder Sockets, wie in Kapitel 2 beschrieben. Der Language Server verwaltet intern ein vollständiges Modell aller CDS-Dateien im Workspace, einschließlich semantischer Informationen wie Typ-Hierarchien, Referenzen und Annotationen. Bei Dateiänderungen aktualisiert der Language Server inkrementell sein internes Modell, wodurch schnelle Antwortzeiten auch bei großen Projekten gewährleistet sind.

5.3.2 workspace/symbol Request im Detail

Für die Service-Struktur-Analyse ist der LSP-Request workspace/symbol relevant. Dieser Request liefert alle Symbole eines Workspace in einem strukturierten JSON. Jedes Symbol enthält den Namen, den Container (übergeordnetes Element), die Datei-URI, die exakte Position (range) im Quellcode sowie den Symboltyp im Feld kind. Quellcode 4 zeigt einen Ausschnitt der Antwort des CAP Language Servers.

```
[
  {
    „name“: „Bookings“,
    „containerName“: „TravelService“,
    „location“: {
      „uri“: „file:///srv/travel-service.cds“,
      „range“: {
        „start“: { „line“: 4, „character“: 0 },
        „end“: { „line“: 21, „character“: 1 }
      }
    },
    „kind“: 5
  },
  {
    „name“: „BookingDate“,
    „containerName“: „TravelService.Bookings“,
    „location“: { ... },
    „kind“: 8
  }
]
```

Quellcode 4 — Beispiel einer Workspace-Symbol-Antwort des CAP Language Servers

Das Feld containerName kodiert die Hierarchie der Artefakte. Ein Wert wie TravelService.Bookings bedeutet, dass das Symbol BookingDate innerhalb von Bookings liegt, welches wiederum zum TravelService gehört. Das Feld kind gibt den Symboltyp an gemäß der LSP-Spezifikation. Die numerischen Werte entsprechen dem SymbolKind-Enum: 5 steht für eine Entity (Typ Class), 8 für ein Feld (Typ Field), 12 für eine Funktion (Typ Function) und 19 für einen Service (Typ Struct). Diese

Information ermöglicht die Rekonstruktion der Service-Hierarchie durch Parsing der containerName-Strings.

Das range-Feld enthält präzise Positionsinformationen mit Start- und End-Position. Jede Position spezifiziert Zeile (line) und Spalte (character), beide nullbasiert. Diese Granularität ermöglicht die exakte Navigation zu jedem Artefakt und erfüllt FA3 vollständig. Im Gegensatz zu CSN sind diese Positionsinformationen für alle Symbole verfügbar, nicht nur für Top-Level-Definitionen.

5.3.3 Integration in VS Code

Der Language Server bietet eine native Integration in VS Code über das LSP. VS Code stellt mit der vscode-languageclient-Bibliothek eine Client-Implementierung bereit, die die Kommunikation mit dem Language Server abstrahiert. Die Extension muss lediglich den Language Server-Prozess starten und die Client-Instanz konfigurieren. Alle LSP-Requests werden automatisch an den Language Server weitergeleitet.

Diese Integration reduziert den Implementierungsaufwand erheblich. Die Extension muss keine eigene Parsing- oder Analyselogik implementieren, sondern nur die vom Language Server bereitgestellten Symbole konsumieren und in einer Tree-View darstellen. Zusätzlich profitiert die Extension von zukünftigen Verbesserungen am Language Server, ohne eigene Anpassungen vornehmen zu müssen. Wenn der Language Server beispielsweise neue CDS-Features unterstützt, sind diese automatisch in der Extension verfügbar.

5.3.4 Erweiterbarkeit und Wartbarkeit

Bei Bedarf ist der Language Server erweiterbar, da diese Arbeit in der Abteilung entsteht, die den CAP Language Server entwickelt. Falls der Request workspace/symbol nicht alle benötigten Informationen liefert, könnten eigene LSP-Requests hinzugefügt werden. Diese würden spezifische Informationen für die servicezentrierte Navigation bereitstellen, etwa bereits aggregierte Service-Strukturen oder zusätzliche Metadaten.

Die Wartbarkeit ist durch die Nutzung eines etablierten Standards hoch. Das LSP ist weitverbreitet und gut dokumentiert. Änderungen an der CDS-Sprache werden zentral im Language Server implementiert und sind automatisch für alle LSP-Clients verfügbar. Die Extension ist von der konkreten Implementierung des CAP-Compilers entkoppelt und kommuniziert nur über die standardisierte LSP-Schnittstelle.

5.3.5 Abhängigkeiten und Voraussetzungen

Die Strategie setzt voraus, dass der CAP Language Server zur Laufzeit verfügbar ist. Dies ist in typischen CAP-Entwicklungsumgebungen der Fall, da der Language Server als Teil der CDS Language Support Extension installiert wird. Ohne verfügbaren Language Server kann die Extension nicht funktionieren. Ein Fallback-Mechanismus auf CSN oder Repository-basiertes Parsing wäre möglich, würde jedoch den Implementierungsaufwand deutlich erhöhen und die Vorteile der Language Server-Integration zunichtemachen.

5.4 Vergleich und Auswahl

Im Rahmen dieser Bachelorarbeit wird nur eine Strategie implementiert. Zur Auswahl wird eine Nutzwertanalyse durchgeführt. Die Strategie mit dem höchsten Nutzwert wird in Kapitel 6 umgesetzt.

5.4.1 Nutzwertanalyse als Bewertungsmethode

Die Nutzwertanalyse ist ein Verfahren zur Bewertung von Entscheidungsalternativen bei mehreren Zielgrößen, bei dem qualitative und quantitative Kriterien systematisch in eine Entscheidung einbezogen werden [48, pp. 193–194]. Das Verfahren eignet sich besonders für Entscheidungsprobleme, bei denen nicht alle Kriterien quantitativ messbar sind, wie es bei der Bewertung von Softwarearchitekturen der Fall ist.

Die Methode funktioniert wie folgt: Zunächst werden relevante Bewertungskriterien definiert und mit Gewichten versehen, die ihre relative Bedeutung ausdrücken. Die Summe aller Gewichte beträgt 1. Jede Alternative wird anschließend bezüglich jedes Kriteriums mit einem Punktwert auf einer definierten Skala bewertet. Der

Gesamtnutzwert einer Alternative ergibt sich aus der Summe der gewichteten Einzelbewertungen. Die Alternative mit dem höchsten Nutzwert wird als vorteilhafteste Option identifiziert [48, pp. 193–194].

Der Gesamtnutzwert N_{Ni} einer Alternative i berechnet sich formal wie folgt [48, p. 196]:

$$N_{Ni} = \sum_{k=1}^K n_{ik} \cdot w_k \quad (1)$$

wobei n_{ik} den Teilnutzenwert der Alternative i bezüglich des Kriteriums k bezeichnet, w_k das Gewicht des Kriteriums k und K die Anzahl der Bewertungskriterien.

Im Folgenden wird die Nutzwertanalyse auf die Bewertung der drei Integrationsstrategien angewendet. Die Gewichtung der Kriterien und die Bewertung der Alternativen erfolgen auf Basis der in Kapitel 4 identifizierten Anforderungen.

5.4.2 Bewertungskriterien und Gewichtung

Für die Nutzwertanalyse werden Bewertungskriterien herangezogen. Diese leiten sich von den Anforderungen ab oder werden als sinnvoll erachtet. Tabelle 4 zeigt diese Bewertungskriterien mit ihrer Gewichtung und Herleitung aus den Anforderungen. In den Abschnitten darunter werden die Kriterien näher erläutert.

Kriterium	Gewicht	Herleitung
K1: Semantische Genauigkeit	25%	FA1, FA2
K2: Positionsinformationen	25%	FA3
K3: VS Code Integration	20%	NFA1
K4: Implementierungsaufwand	15%	–
K5: Wartbarkeit	10%	NFA4
K6: Unabhängigkeit	5%	–

Tabelle 4 — Bewertungskriterien und Gewichtung

Die Gewichtung orientiert sich an der Kritikalität für die Funktionsfähigkeit der Extension. Kriterien, ohne die die Extension nicht nutzbar wäre (K1, K2), erhalten die höchste Gewichtung von jeweils 25%. Kriterien, die die Qualität der Lösung beeinflussen (K3, K4, K5), erhalten mittlere Gewichtungen zwischen 10% und 20%.

Kriterien, die Abhängigkeiten betreffen, aber durch höhere Funktionalität kompensiert werden können (K6), erhalten die niedrigste Gewichtung von 5%.

K1: Semantische Genauigkeit (25%) beschreibt die Fähigkeit, CDS-Artefakte korrekt zu identifizieren und zu interpretieren. Das Kriterium leitet sich aus FA1 und FA2 ab, da Services korrekt erkannt werden müssen. Es erhält die höchste Gewichtung, da ohne korrekte Erkennung die Extension nicht nutzbar ist.

K2: Positionsinformationen (25%) bezeichnet die Verfügbarkeit von Quellcode-Positionen (Zeile/Spalte) für Artefakte. Das Kriterium leitet sich aus FA3 ab, da Navigation zu Definitionen möglich sein muss. Es erhält ebenfalls die höchste Gewichtung, da Navigation das zentrale Feature der Extension ist.

K3: VS Code Integration (20%) bewertet die Kompatibilität mit VS Code APIs und der Erweiterungsarchitektur. Das Kriterium leitet sich aus NFA1 ab und fordert eine nahtlose Einbindung in die IDE. Die hohe Gewichtung ergibt sich daraus, dass schlechte Integration die Akzeptanz mindert.

K4: Implementierungsaufwand (15%) erfasst Komplexität und Zeitbedarf für die Umsetzung. Es gibt keine direkte Anforderung, jedoch besteht eine praktische Ressourcenbeschränkung im Rahmen dieser Bachelorarbeit. Die mittlere Gewichtung spiegelt den realistischen Projektrahmen wider.

K5: Wartbarkeit (10%) bewertet die langfristige Anpassbarkeit bei Änderungen am CAP-Framework. Das Kriterium leitet sich aus NFA4 ab, da die Extension auch zukünftig funktionieren soll. Die geringere Gewichtung ergibt sich daraus, dass zunächst die Funktionalität wichtiger ist.

K6: Unabhängigkeit (5%) bezeichnet die Freiheit von externen Laufzeitabhängigkeiten. Es gibt keine direkte Anforderung. Abhängigkeiten sind akzeptabel, wenn die Funktionalität dadurch besser wird. Die niedrigste Gewichtung ist ein bewusster Trade-off.

Tabelle 5 zeigt die Bewertung der drei Strategien anhand der definierten Kriterien.

Kriterium	Gewicht	Repository	CSN	Language Server
K1: Semantische Genauigkeit	25%	2	5	5
K2: Positionsinformationen	25%	3	1	5
K3: VS Code Integration	20%	2	3	5
K4: Implementierungsaufwand	15%	2	4	3
K5: Wartbarkeit	10%	2	3	4
K6: Unabhängigkeit	5%	5	2	2
Gewichtete Summe	100%	2.40	3.10	4.45

Tabelle 5 — Nutzwertanalyse der Integrationsstrategien

Punkteskala: 1 = unzureichend | 2 = ausreichend | 3 = befriedigend | 4 = gut | 5 = sehr gut

Die detaillierte Begründung der Punktvergabe für jedes Kriterium und jede Strategie ist in Tabelle 13 im Anhang dokumentiert. Die wesentlichen Unterschiede lassen sich wie folgt zusammenfassen. Bei K1 (Semantische Genauigkeit) erzielen CSN und Language Server Höchstwerte, da beide den CAP-Compiler nutzen. Der entscheidende Differenzierungsfaktor ist K2 (Positionsinformationen). CSN liefert diese nur unvollständig, während der Language Server über die Location-API präzise Quellcode-Positionen bereitstellt. Bei K3 (VS Code Integration) profitiert der Language Server von der nativen LSP-Unterstützung, während Repository und CSN zusätzliche Adapter erfordern. Der Implementierungsaufwand (K4) ist bei CSN am geringsten, da das JSON-Format direkt parsebar ist. Bei K5 (Wartbarkeit) punktet der Language Server durch die Stabilität des LSP-Standards, während ein eigener Parser bei jeder Sprachänderung angepasst werden müsste. K6 (Unabhängigkeit) zeigt den Trade-off: Die Repository-Strategie ist vollständig unabhängig, während CSN und Language Server externe Abhängigkeiten mitbringen.

5.5 Auswahl der Integrationsstrategie

Die Nutzwertanalyse ergibt den höchsten Wert für die Language Server-basierte Integration mit 4.45 Punkten (siehe Abbildung 5). Der Language Server bietet die beste VS Code Integration, da er das LSP nativ implementiert. Er liefert vollständige Positionsinformationen, während CSN diese nur unvollständig bereitstellt. Ein eigener Parser wäre aufwendig und fehleranfällig. Zusätzlich kann der Language

Server bei Bedarf erweitert werden, da diese Arbeit in der Abteilung entsteht, die ihn entwickelt.

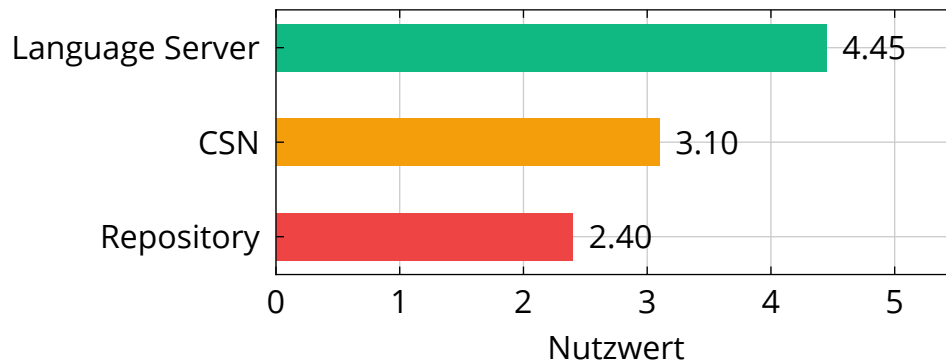


Abbildung 5 — Ergebnis der Nutzwertanalyse

Die Wahl des Language Servers bringt eine Abhängigkeit zur Laufzeit mit sich. Dieser Trade-off wird zugunsten der höheren Funktionalität akzeptiert. Kapitel 6 beschreibt die Implementierung der Extension auf Basis der gewählten Language Server-Integration.

6 Implementierung des Prototyps

Im folgenden Kapitel wird ein Prototyp basierend auf der Language Server-basierten Strategie implementiert. Der Fokus liegt auf einer aggregierten Service-Übersicht zur Navigation sowie einer Suchfunktion mit Filteroptionen.

6.1 Architekturkonzept und Designentscheidungen

Die Extension folgt einer Mehrschichtigen Architektur zur Trennung von Datenabfrage, Business Logic und UI-Präsentation [49, Sec. 2]. Die folgenden Abschnitte beschreiben den strukturellen Aufbau und zentrale Designentscheidungen.

6.1.1 Architektur

Abbildung 6 zeigt die Komponentenstruktur. Das Modul `extension.ts` orchestriert vier Hauptkomponenten. Der `cdsClient` beschafft Daten vom CDS Language Server, der Controller verwaltet Zustand und koordiniert die Business Logic, der `TreeProvider` rendert die UI, und der `SearchProvider` stellt die Suchoberfläche bereit.

Der Domain Layer (`treeBuilder`, `search`, `filter`) enthält die Business Logic für Datentransformation und Filterung. Diese Schicht hat keine VS Code Abhängigkeiten und ist isoliert testbar. Der Datenfluss verläuft vom Language Server über den Controller (mit Domain-Modulen) zum `TreeProvider`. User Input fließt vom `SearchProvider` zurück zum Controller.

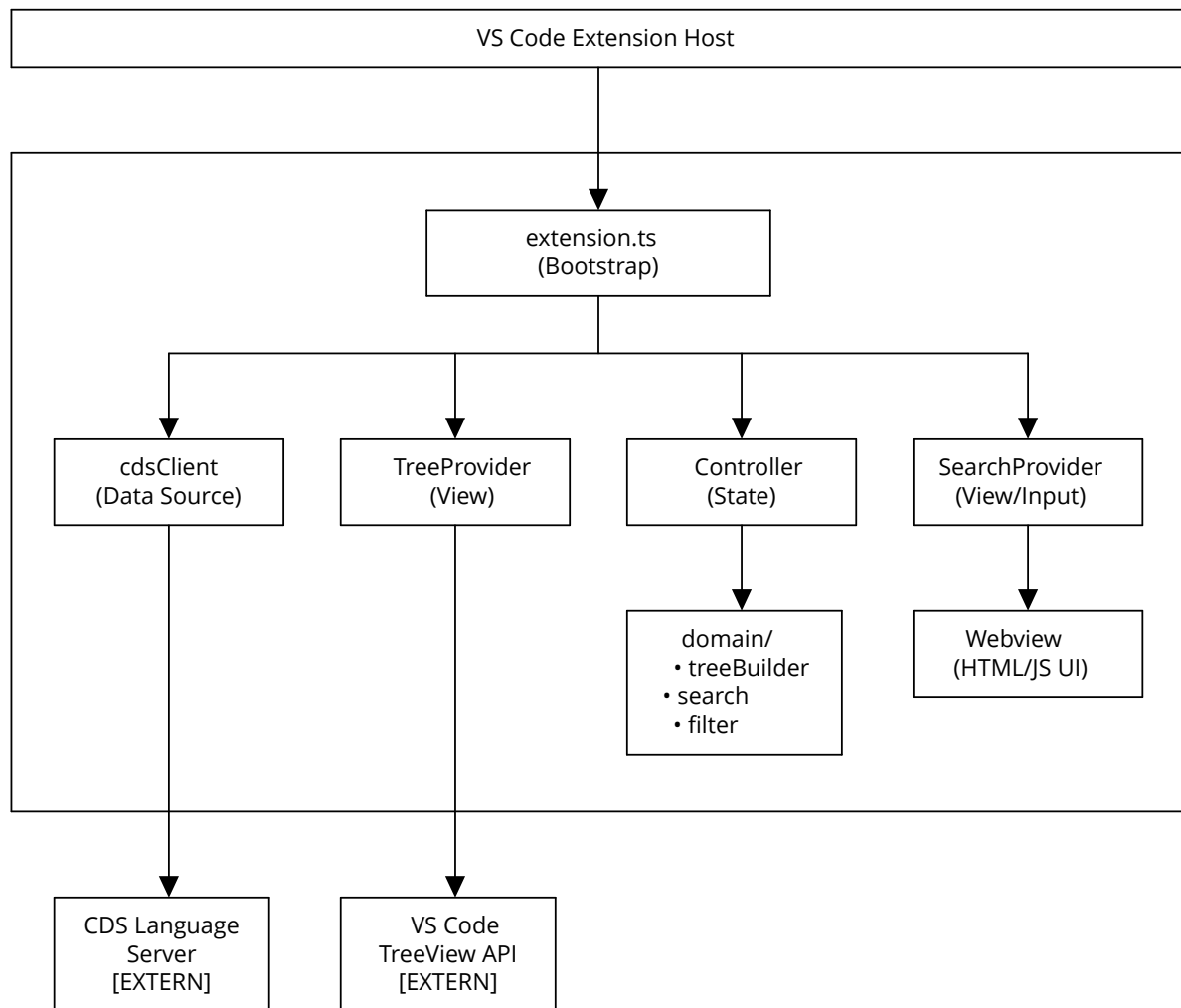


Abbildung 6 — Komponentenstruktur und Datenfluss der VS Code Extension

6.1.2 Technologie-Stack

Die Extension wurde in TypeScript implementiert, was Typsicherheit zur Compile-Zeit und vollständige IDE-Unterstützung bietet. Das Suchpanel wurde als Webview mit nativem HTML, CSS und JavaScript realisiert, da VS Code's native UI-Komponenten für die komplexen Anforderungen (Suchfeld, Filterchips, Optionen) zu limitiert sind. Die Webview sendet SearchMessage-Events (queryChanged, submit, clear) an die Extension, die diese in FilterSpec-Objekte umwandelt und an den Controller weiterleitet. Auf Frontend-Frameworks wurde verzichtet, um Bundle-Overhead zu vermeiden. Der Build-Prozess wird über npm [50] gesteuert, die Extension als VSIX-Datei ausgeliefert.

6.2 Integration mit CDS Language Server

Die Extension nutzt den CDS Language Server als zentrale Datenquelle für Service-Strukturinformationen. Dieser Abschnitt beschreibt die Kommunikation über das LSP und die Verarbeitung der empfangenen Symboldaten.

6.2.1 Kommunikation über Language Server Protocol

Die Extension nutzt den `workspace/symbol` Request des LSP als zentrale Datenquelle. Die Grundlagen des LSP wurden in Kapitel 2 erläutert. Der Request akzeptiert einen optionalen Query-Parameter, mit dem das Ergebnis gefiltert und erweitert werden kann. Die Extension verwendet die Query `/gax`, wobei jeder Buchstabe eine spezifische Option aktiviert. `g` inkludiert Abhängigkeiten aus `node_modules`, `a` inkludiert automatisch im Service bereitgestellte Entities und `x` kennzeichnet Explorer-Queries.

Das `x`-Flag wurde im Rahmen dieser Arbeit speziell für die Extension entwickelt und vom Language Server-Team der Abteilung implementiert [51]. Dieser Beitrag zum CDS Language Server ermöglicht die servicezentrierte Navigation überhaupt erst, da die Standard-LSP-Response diese Kategorisierung nicht bereitstellt. Der Language Server reichert die Response um essenzielle Metadaten an: die Kennzeichnung von Domain-Model-Entities, Flags für automatisch bereitgestellte Entities sowie Hierarchie-Informationen zur Unterscheidung von Service-Projections und Datenbank-Entities. Ohne diese Erweiterung müsste die Extension auf die in Kapitel 5 abgelehnte Repository-basierte Strategie zurückfallen. Die Extension ist als internes SAP-Projekt verfügbar [3].

6.2.2 Normalisierung der Symboldaten

Die Response des `workspace/symbol` Requests liefert für jedes Symbol Metadaten im LSP-Format. Die detaillierte Response-Struktur ist in Anhang A dokumentiert. Die Extension normalisiert diese Rohdaten zu einem einheitlichen `Symbol` Interface, das die Felder `name`, `kind`, `containerName`, `fileUri`, `range` und `flags` enthält. Die numerischen Werte werden über Konstanten interpretiert, etwa Typ 5 wird zu ENTITY.

Das zentrale Merkmal dieser Response ist das `flags`-Feld, das nur durch das `x`-Flag in der `/gax` Query aktiviert wird. Dieses Flag wurde speziell für die Extension entwickelt, um die Response um essenzielle Metadaten zur servicezentrierten Navigation anzureichern. Die Flags sind als Integer kodiert, wobei jedes Bit eine spezifische Information repräsentiert. Dies ermöglicht die effiziente Kombination mehrerer Eigenschaften in einem einzigen Zahlenwert. Die Flags kodieren Metadaten zur Kategorisierung von Symbolen, beispielsweise `DOMAIN_MODEL` für Entities im Datenbank-Schema, `EXTERNAL_SERVICE` für externe Services, `AUTOEXPOSED_ENTITY` für automatisch im Service bereitgestellte Entities sowie `VIEW` zur Unterscheidung von Projections und persistenten Entities. Darüber hinaus liefern die Flags weitere für die Extension relevante Informationen wie `ASSOCIATION` und `COMPOSITION` zur Kennzeichnung von Beziehungstypen, `ASPECT` für wiederverwendbare Modellfragmente sowie `EXTEND` zur Identifikation erweiterter Definitionen. Die vollständige Flag-Spezifikation mit allen 22 definierten Flags ist in Anhang B dokumentiert. Ohne diese Flags wäre eine Unterscheidung zwischen Domain-Entities und Service-Projections nicht möglich, da beide als `Typ=ENTITY` klassifiziert sind.

6.2.3 Erweiterung um zusätzliche LSP-Features

Die Extension nutzt neben `workspace/symbol` weitere LSP-Operationen für IDE-Funktionen. Diese sind über das Context-Menü der TreeView-Items zugänglich und erweitern die Navigation um typische IDE-Features wie Referenzsuche, Definition-Lookup und Implementation-Mapping.

Über Rechtsklick auf ein CDS-Artefakt im Tree kann der User „Go to Definition“ oder „Find References“ ausführen. Diese Befehle delegieren an den CDS Language Server via `vscode.commands.executeCommand`. Find References öffnet die References-View von VS Code mit allen Fundstellen des Symbols und ist besonders wertvoll bei Refactorings oder beim Verstehen von Abhängigkeiten zwischen Services.

Der „Go to Implementation“-Befehl findet Event Handler-Dateien (`.js/.ts`) für Services. CAP-Services bestehen aus CDS-Definitionen und optionalen Implementierungsdateien mit registrierten Event Handlers. Die Extension sendet eine `textDocument/implementation` Request an den Language Server, der die URIs der

Handler-Dateien zurückgibt. Dieses Feature ist besonders nützlich, da Implementierungsdateien nicht durch Namenskonventionen auffindbar sind. Der Language Server analysiert die `require`-Statements und registrierte Handler im Projekt, um die Zuordnung zwischen Service-Definition und Implementierung zu ermitteln.

6.2.4 Automatische Entdeckung von Implementierungsdateien

Wie in Kapitel 4 beschrieben, sind die Artefakte eines CAP-Services über mehrere Dateien verteilt. Besonders die Zuordnung von Event-Handler-Implementierungen zu Services ist nicht trivial, da keine eindeutige Namenskonvention existiert. Ein Service `CatalogService` kann beispielsweise in `catalog-service.js`, `catalog.js` oder `CatalogService.ts` implementiert sein.

Die Extension löst dieses Problem durch automatische Entdeckung via Language Server. Beim Expandieren eines Service-Nodes im Tree fragt die Extension den CDS Language Server nach zugehörigen Implementierungsdateien. Der Language Server analysiert die Event-Handler-Registrierungen im Projekt und liefert alle relevanten Dateien zurück. Diese werden im „Files“-Ordner unter dem jeweiligen Service angezeigt. Die Ergebnisse werden gecacht, um die Performance zu optimieren.

Dieses Feature arbeitet lazy: Implementierungsdateien werden erst bei Bedarf geladen, wenn der User einen Service expandiert. Dies reduziert die initiale Ladezeit der Extension, da nicht für alle Services sofort Implementation-Lookups durchgeführt werden müssen. Die asynchrone Natur der Implementierung stellt sicher, dass die UI responsiv bleibt, auch wenn der Language Server mehrere Sekunden für die Analyse benötigt.

6.3 Datenmodell und Baumaufbau

Die flache Liste von Symbolen aus dem Language Server muss in eine hierarchische Baumstruktur transformiert werden, die in VS Code's TreeView dargestellt werden kann. Dieser Abschnitt beschreibt das zugrunde liegende Datenmodell und die gewählte Navigationsstruktur.

6.3.1 TreeNode-Datenmodell

Der Language Server liefert Symbole als flache Liste, in der jedes Symbol unabhängig mit seinen Metadaten repräsentiert ist. Diese Repräsentation ist für die Analyse geeignet, aber nicht für eine hierarchische UI-Darstellung. Die Extension transformiert daher die Symbole in eine Baumstruktur aus `TreeNode`-Objekten, die zusammengehörige Artefakte aggregiert, hierarchische Beziehungen etabliert, und UI-spezifische Informationen ergänzt.

```
export interface TreeNode {  
  label: string  
  children?: TreeNode[];  
  nodeType?: NodeType  
  fileUri?: string  
  range?: Range  
  serviceName?: string  
}
```

Quellcode 5 — `TreeNode` Interface

Das `TreeNode`-Interface bildet die Grundlage der Baumdarstellung. Das Feld `nodeType` bestimmt die visuelle Repräsentation (Icons, Farben) und unterscheidet zwischen verschiedenen Artefakt-Typen wie Services, Entities, Actions oder Ordnern. Die Felder `fileUri` und `range` ermöglichen die Navigation zur Quellcode-Position, während `serviceName` servicebasierte Filterung unterstützt.

6.3.2 Aufbau der Navigationsstruktur

Die Extension implementiert eine servicezentrierte Navigationsstruktur, die das in Kapitel 4 identifizierte Problem der verteilten Service-Artefakte adressiert. Die gewählte Struktur orientiert sich an der fachlichen Gliederung von CAP-Projekten und trennt zwischen Service-spezifischen Artefakten und domänenübergreifenden Modellen.

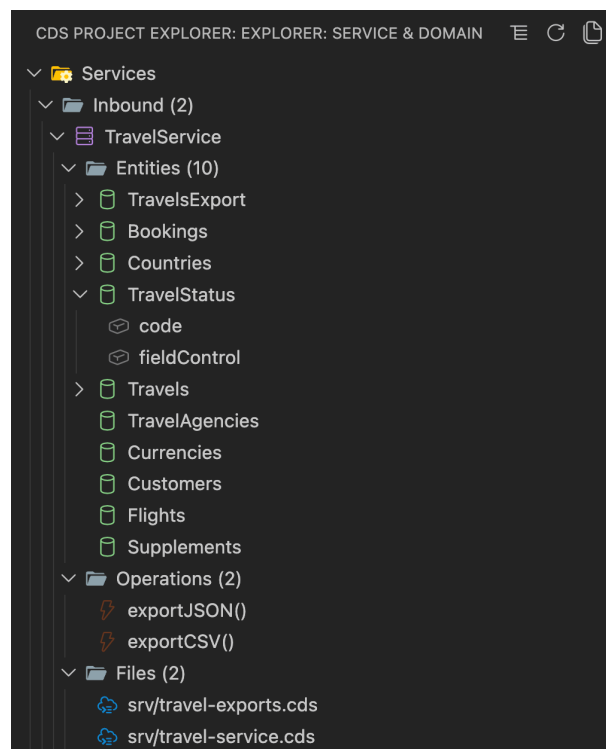


Abbildung 7 — Navigationsstruktur der Extension [3]

Der Services-Bereich bildet den Hauptbereich der Extension und unterteilt sich in **Outbound** und **Inbound** Services. Outbound-Services repräsentieren externe APIs oder Remote Services, während Inbound-Services die Kernlogik der Anwendung bereitstellen. Die Unterscheidung erfolgt anhand des `EXTERNAL_SERVICE` Flags, das in Kapitel 6.2.2 beschrieben wurde. Services mit diesem Flag werden als Outbound klassifiziert. Diese Unterscheidung ist für die Navigation relevant, da externe Services typischerweise nur konsumiert werden, während interne Services implementiert und erweitert werden.

Innerhalb eines Services erfolgt eine Kategorisierung nach Artefakt-Typen (z.B. Entities, Operations, Files). Die Aggregation der Struktur erfolgt über die hierarchische Information im `containerName`-Feld, wobei die Notation „TravelService.Travels.bookings“ in die entsprechende Baum-Hierarchie überführt wird.

6.4 Navigation und Interaktion

Die Visualisierung erfolgt über VS Code's TreeView API (siehe Kapitel 2). Die Extension transformiert die `TreeNode` Objekte in darstellbare Elemente und registriert Click-Handler für die Navigation zum Quellcode.

Ein Controller verwaltet den Zustand der Ansicht, wobei zwischen Tree-Modus (vollständiger Baum) und Filter-Modus (gefilterte Ansicht) unterschieden wird. Der Controller koordiniert die Interaktion zwischen TreeView und Suchfunktion.

Das Such- und Filtersystem kombiniert Textsuche mit strukturierter Filterung nach Artefakt-Typen. Spezielle Token wie `@entity` oder `@service` ermöglichen die gezielte Filterung, die mit Freitextsuche kombiniert werden kann (z.B. `@entity booking`).

Die Filterlogik implementiert einen zweistufigen rekursiven Algorithmus. Im ersten Durchlauf prüft die Extension für jeden Node, ob er selbst die Filterkriterien erfüllt oder ob seine Kinder Treffer enthalten. Im zweiten Durchlauf werden zusätzlich alle Geschwister-Nodes eines Treffers angezeigt (Sibling-Context-Preservation). Dies verhindert, dass der User beim Filtern die Orientierung verliert, da nicht-passende Geschwister-Nodes als Kontext erhalten bleiben. Passende Nodes werden in einem `autoExpandedIds`-Set gespeichert, damit der `TreeProvider` sie automatisch aufklappt. Übergeordnete Nodes bleiben sichtbar, solange ihre Kinder Treffer enthalten. Treffer werden im TreeView hervorgehoben.

6.4.1 State Management und Controller-Pattern

Die Extension implementiert das Controller-Pattern zur Zentralisierung der Zustandsverwaltung. Der Controller hält den `ViewState` (Such-Query, Anzeigemodus, expandierte Nodes) und koordiniert die Interaktion zwischen `TreeProvider` und `SearchProvider`. Diese Entkopplung ermöglicht es, State-Logik unabhängig von der VS Code API zu testen.

Das `ViewState`-Objekt unterscheidet zwischen zwei Modi: Im Tree-Modus zeigt die Extension den vollständigen Baum, wobei ein `userExpandedIds`-Set die manuell expandierten Nodes trackt. Im Filter-Modus wird nur der gefilterte Baum angezeigt, wobei ein `searchExpandedIds`-Set die automatisch expandierten passenden Nodes

enthält. Diese Trennung verhindert, dass automatische Expansion beim Filtern die User-Präferenzen im Tree-Modus überschreibt.

Der Controller implementiert das Observer-Pattern via Callbacks. Bei State-Änderungen (z.B. `setFilterSpec`, `setStrategy`) notifiziert er den TreeProvider via `onStateChange`-Callback, der daraufhin die UI refreshed. Der TreeProvider wiederum informiert den Controller via `onDidChangeData`-Callback, wenn neue Symboldaten vom Language Server geladen wurden. Diese bidirektionale Kommunikation stellt sicher, dass UI und State stets synchron bleiben.

6.5 Fehlerbehandlung und Benutzer-Feedback

Die Extension implementiert robuste Fehlerbehandlung für verschiedene Fehler-szenarien bei der LSP-Kommunikation. Eine eigene Error-Klasse `CdsClientError` mit den Feldern `code`, `message` und `cause` ermöglicht differenzierte Reaktionen auf unterschiedliche Fehlerfälle. Tabelle 6 gibt eine Übersicht aller Error-Typen.

Error-Typ	Ursache	Anzeige im Tree	Benutzeraktion
Installation Error	CDS Extension ist nicht installiert	Error-Node mit Installations-Link	Installation über VS Code Marketplace
Version Error	CDS Extension Version < 9.9.0	Error-Node mit Versionshinweis	Extension aktualisieren
LSP Request Error	workspace/symbol Request fehlgeschlagen	Error-Node mit Fehlermeldung als Tooltip	„Show Output“ für vollständigen Log
Language Server Inactive	Language Server ist nicht aktiviert oder bereit	Error-Node mit Hinweis auf Aktivierung	Warten oder VS Code neu starten
Timeout Error	LSP Request überschreitet Timeout-Grenze	Error-Node mit Timeout-Meldung	Projekt verkleinern oder Timeout erhöhen

Tabelle 6 — Error-Typen und deren Behandlung in der Extension

Die Extension unterscheidet zwischen Installationsfehlern der erforderlichen CDS Extension, Versionskonflikten und fehlgeschlagenen LSP-Requests. Bei Installationsfehlern wird ein Error-Node mit Installations-Link zum VS Code Marketplace angezeigt. Die Versionsprüfung stellt sicher, dass mindestens Version 9.9.0 installiert ist, da das `/gax`-Flag erst ab dieser Version verfügbar ist. Bei fehlgeschlagenen

LSP-Requests zeigt die Extension die Fehlermeldung als Tooltip an und ermöglicht den Zugriff auf den vollständigen Log über den „Show Output“-Befehl.

Während des Ladevorgangs informiert die Extension den User über den aktuellen Fortschritt durch eine Sequenz von Loading-Messages im Tree, beispielsweise „Activating CDS Extension“, „Querying workspace symbols“, „Processing symbols“ und „Building tree“. Diese Granularität ist notwendig, da der workspace/symbol Request bei großen Projekten mehrere Sekunden dauern kann. Die Status-Messages verhindern die Wahrnehmung eines eingefrorenen UIs und kommunizieren, dass die Extension aktiv arbeitet. Nach erfolgreichem Laden verschwindet der Loading-Node und der vollständige Tree wird angezeigt.

6.6 Ergebnis der Implementierung

Die implementierte Extension integriert sich als TreeView in VS Code's Sidebar und aggregiert alle Artefakte eines Service in einer navigierbaren Baumstruktur. Ein Klick auf ein Element im Tree öffnet die entsprechende Datei und markiert die relevante Stelle im Code. Abbildung 8 zeigt die Extension im Einsatz.

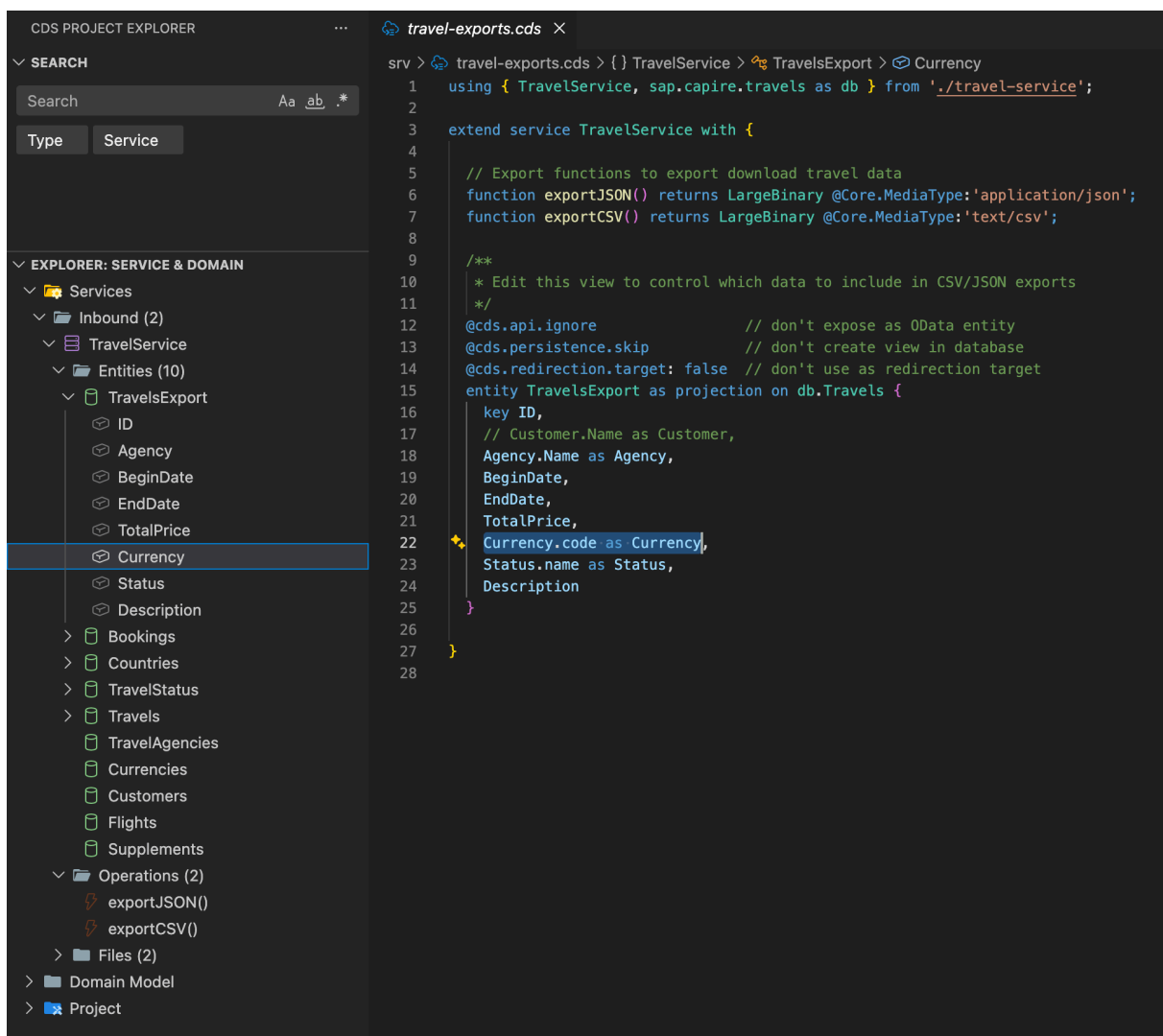


Abbildung 8 — Extension mit TreeView und Code-Navigation [3]

Die Suchfunktion am oberen Rand des TreeView ermöglicht die Filterung der Artefakte. Die Extension filtert den Baum und zeigt nur die relevanten Treffer an, während die hierarchische Struktur erhalten bleibt. Abbildung 9 zeigt die Suche nach dem Begriff „travel“.

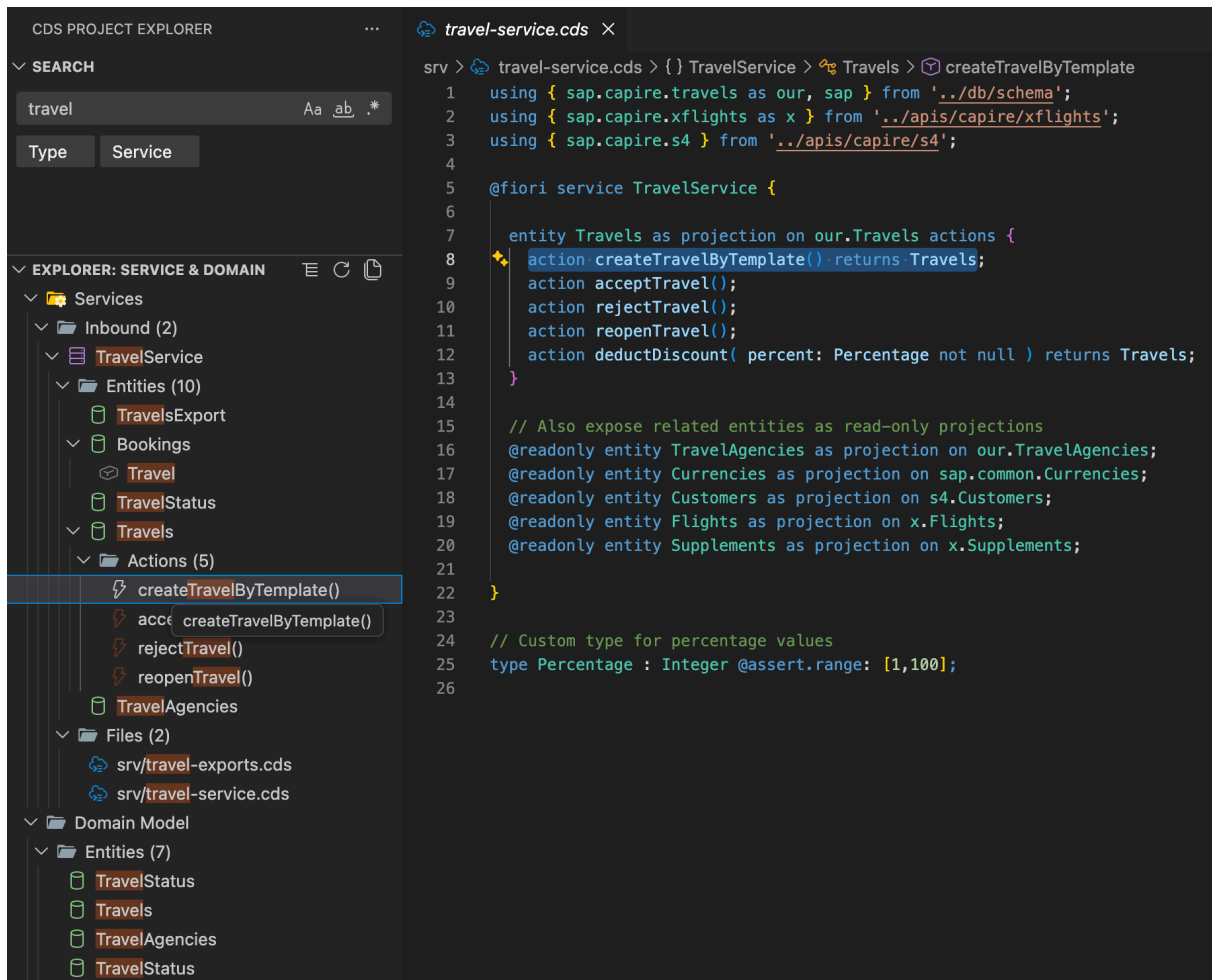


Abbildung 9 — Suchfunktion mit gefilterter Baumansicht [3]

Zusätzlich zur Textsuche unterstützt die Extension strukturierte Filterung nach Artefakt-Typen. Die Filter können kombiniert werden, um gezielt bestimmte Artefakt-Typen anzuzeigen und andere auszublenden. Abbildung 10 zeigt die Filterung mit speziellen Token.

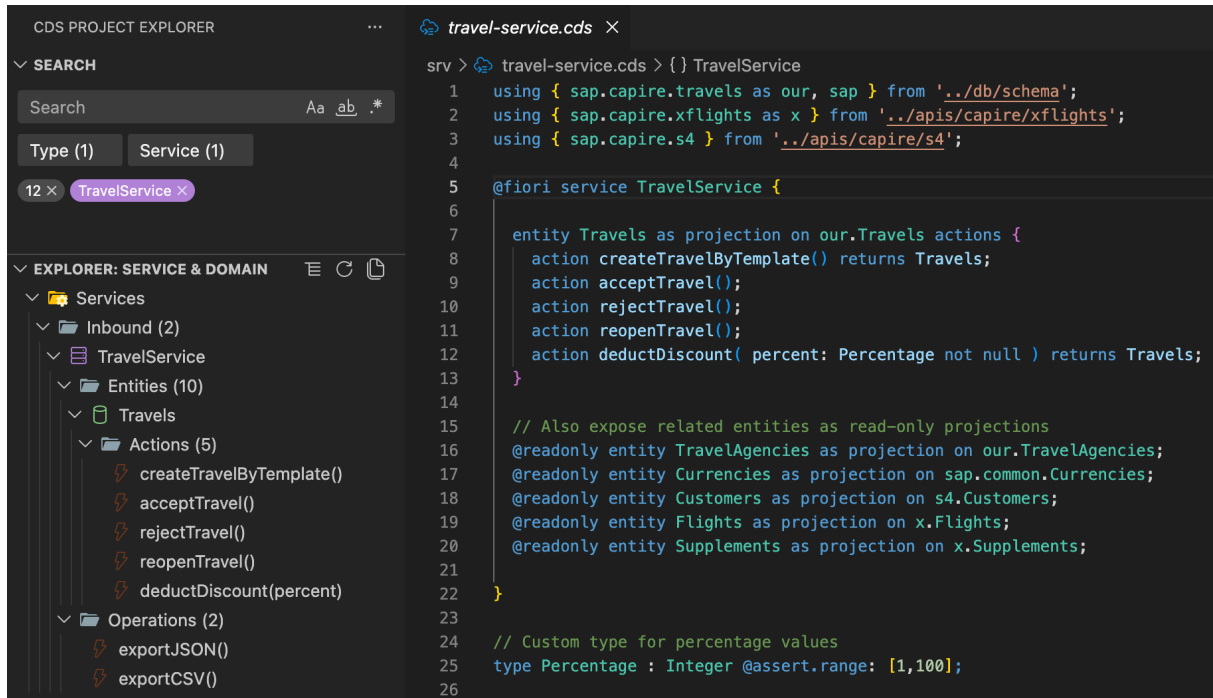


Abbildung 10 — Filterung nach Artefakt-Typen [3]

7 Evaluation

Die Evaluation bildet die fünfte Phase des DSR-Prozesses nach Peffers et al. [11]. Dieses Kapitel prüft, inwieweit die Extension die in Kapitel 4 definierten Anforderungen erfüllt. Die Bewertung erfolgt kriterienbasiert anhand von drei Testprojekten unterschiedlicher Größe und Komplexität.

7.1 Testprojekte

Die Extension wurde an drei CAP-Projekten evaluiert, die sich in Größe und Komplexität unterscheiden.

bookshop ist das offizielle CAP-Referenzbeispiel von SAP [6]. Das Projekt eignet sich als Baseline, da es die dokumentierte Best-Practice-Struktur abbildet und als kleines, übersichtliches Projekt den unteren Komplexitätsbereich repräsentiert.

xtravels ist ein Travel-Management-System mittlerer Komplexität [52]. Das Projekt eignet sich, weil es die in Kapitel 4 beschriebene Fragmentierung zeigt. Service-Definition, Handler-Implementierung und Authorization-Annotationen liegen in separaten Dateien.

ctsm-develop ist ein produktives Clinical-Trial-System [53]. Das Projekt eignet sich, weil es die Komplexität realer Enterprise-Anwendungen abbildet. Es umfasst mehrere Services, verschachtelte Verzeichnisstrukturen und externe Service-Referenzen.

Diese Auswahl deckt kleine, mittlere und große CAP-Projekte ab und testet die Extension unter unterschiedlichen Bedingungen.

Projekt	Services	Entities	CDS-Dateien	Kategorie
bookshop	2	4	16	Klein
xtravels	3	9	16	Mittel
ctsm-develop	66	1835	803	Groß

Tabelle 7 — Charakterisierung der Testprojekte nach Größenmetriken

Die Kategorisierung basiert auf der Anzahl von Services und Entities. Kleine Projekte (< 3 Services, < 5 Entities) repräsentieren einfache Anwendungsfälle und dienen als Baseline. Mittlere Projekte (1–10 Services, 5–20 Entities) entsprechen typischen Entwicklungsprojekten. Große Projekte (> 50 Services, > 1000 Entities) bilden produktive Enterprise-Anwendungen mit komplexen Datenmodellen ab.

7.2 Vergleich mit bestehenden Ansätzen

Die Extension wird mit drei bestehenden Navigationsmethoden verglichen: VS Code Outline-View, File Explorer und direkte Nutzung der CDS Language Server-Features (Go to Definition, Find References).

Die Outline-View zeigt nur Symbole der aktuell geöffneten Datei und bietet keine servicezentrierte Aggregation über Dateigrenzen hinweg. Ein Service, dessen Artefakte über db/, srv/ und app/ verteilt sind, ist in der Outline nicht als Einheit erfassbar. Die Extension adressiert dies durch projektweite Aggregation aller servicebezogenen Artefakte in einer einheitlichen Baumstruktur.

Der File Explorer navigiert entlang der physischen Ordnerstruktur. Zusammengehörige Service-Artefakte müssen manuell über mehrere Ordner hinweg gesucht werden. Die Extension invertiert diese Perspektive: Der Service ist der primäre Einstiegspunkt, die Dateien sind sekundär im „Files“-Ordner sichtbar. Dies entspricht dem fachlichen mentalen Modell von CAP-Entwicklern, die in Services denken, nicht in Ordnern.

Die direkten Language Server-Features (Go to Definition, Find References) unterstützen punktuelle Navigation von einem Symbol zu dessen Verwendungsstellen. Eine Übersicht aller Service-Komponenten ohne initialen Einstiegspunkt ist nicht möglich. Die Extension kombiniert beide Ansätze: Übersicht (Tree) plus Detail-Navigation (LSP-Features im Context-Menü).

Die Extension ersetzt diese Werkzeuge nicht, sondern ergänzt sie um die fehlende servicezentrierte Perspektive. Die bestehenden Tools bleiben für ihre spezifischen

Anwendungsfälle (datei-lokale Navigation, Verzeichnis-Browsing, Symbol-zu-Symbol-Navigation) weiterhin relevant.

7.3 Ergebnisse der Evaluation

FA1 Identifikation von CAP-Services. Die Extension erkennt Services automatisch in allen drei Testprojekten. Nach dem Öffnen eines Projekts werden die Services in der Sidebar unter „Services“ angezeigt und nach Inbound und Outbound kategorisiert.

FA2 Aggregierte Serviceübersicht. Alle zu einem Service gehörenden Artefakte werden hierarchisch gruppiert dargestellt. Entities, Actions, Operations und zugehörige Dateien sind unter dem jeweiligen Service sichtbar. Die Aggregation ersetzt die manuelle Rekonstruktion der Service-Struktur über mehrere Dateien.

FA3 Navigation zu Artefakten. Ein Klick auf ein Artefakt im Tree öffnet die entsprechende Datei und positioniert den Cursor an der exakten Zeile der Definition. Die Positionsinformationen stammen vom Language Server und sind präzise.

FA4 Darstellung von Artefaktbeziehungen. Die Extension zeigt hierarchische Beziehungen zwischen Artefakten. Ein Service enthält Entities, eine Entity enthält Actions und Fields. Referenzielle Associations zwischen Entities werden jedoch nicht dargestellt. Ein Feld wie Agency erscheint als normales Feld, ohne dass erkennbar ist, dass es auf eine andere Entity verweist. Dieser Limitation liegt zugrunde, dass der Language Server über `workspace/symbol` keine Typ-Auflösung für Felder bereitstellt. Eine Erweiterung würde zusätzliche LSP-Requests (`textDocument/definition` für jedes Feld) erfordern, was die Ladezeit signifikant erhöhen würde. Dies wurde als Trade-off zugunsten der Performance nicht implementiert.

NFA1 Integration in Visual Studio Code. Die Extension ist als natives VS Code Plugin implementiert und nutzt die TreeView API für die hierarchische Darstellung. Die Installation erfolgt über eine VSIX-Datei.

NFA2 Kompatibilität mit CAP-Projektstrukturen. Alle drei Testprojekte folgen dem CAP-Standardlayout und werden korrekt erkannt. Die Extension analysiert die Projektstruktur automatisch.

NFA3 Performante Analyse. Abbildung 11 zeigt die gemessenen Ladezeiten beim initialen Projektöffnen. Die Messungen erfolgten isoliert mit nur CDS Language Server und Extension unter den im Anhang dokumentierten Testbedingungen (siehe Tabelle 10). Jedes Projekt wurde dreimal geöffnet (VS Code Neustart zwischen Messungen), die angegebenen Werte sind Durchschnittswerte. Die Messung umfasst die Zeit von „Extension aktiviert“ bis „Tree vollständig gerendert“. Alle Projekte laden deutlich unter der 10-Sekunden-Schwelle (0.56s - 1.86s). Alle nachfolgenden Navigationsoperationen im Tree erfolgen unmittelbar (< 100ms), da die Datenstruktur bereits im Speicher liegt und lediglich UI-Updates erforderlich sind.

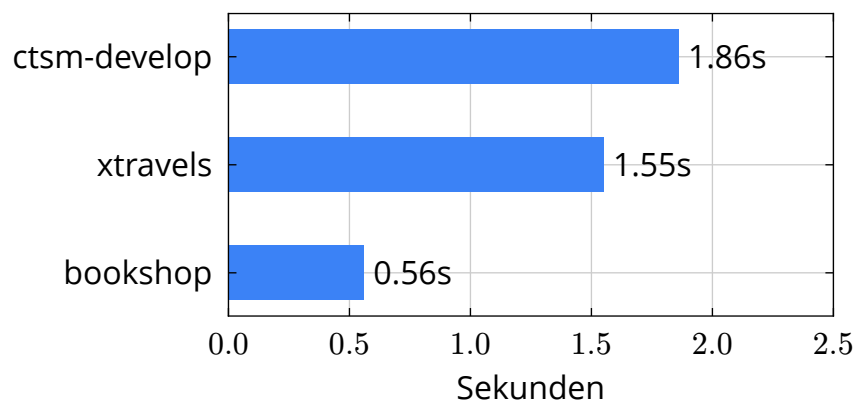


Abbildung 11 — Ladezeiten der Extension nach Projektöffnung

NFA4 Aktualität der Artefaktübersicht. Nach dem Speichern einer geänderten CDS-Datei aktualisiert sich der Tree nicht automatisch. Ein manueller Refresh ist erforderlich. Die automatische Aktualisierung wurde im Rahmen dieser Arbeit als nicht kritisch priorisiert und kann in zukünftigen Versionen ergänzt werden, sobald der Language Server entsprechende Optimierungen für schnellere Refresh-Zyklen bereitstellt.

Tabelle 8 zeigt die Reduktion des Navigationsaufwands anhand der in Kapitel 4 definierten Szenarien. Die detaillierten Navigationsabläufe sind im Anhang dokumentiert.

Szenario	Projekt	Ohne Extension	Mit Extension
Service verstehen	bookshop	3 Dateien, 9 Schritte	2 Klicks im Tree
	xtravels	4 Dateien, 12 Schritte	3 Klicks im Tree
	ctsm-develop	4 Dateien, 12 Schritte	2 Klicks im Tree
Action lokalisieren	bookshop	2 Dateien, 6 Schritte	2 Interaktionen
	xtravels	3 Dateien, 9 Schritte	2 Interaktionen
	ctsm-develop	3 Dateien, 9 Schritte	2 Interaktionen
Entities identifizieren	bookshop	2 Dateien, 6 Schritte	1 Klick
	xtravels	3 Dateien, 9 Schritte	2 Klicks
	ctsm-develop	3 Dateien, 9 Schritte	1 Klick

Tabelle 8 — Vergleich des Navigationsaufwands

Die Extension reduziert den Navigationsaufwand in den evaluierten Szenarien um durchschnittlich 79% über drei Projekte und neun Szenarien hinweg. Die Aussagekraft dieser Metrik ist bei einer Stichprobe von drei Projekten begrenzt, eine statistische Generalisierbarkeit kann nicht beansprucht werden. Tabelle 9 fasst die Ergebnisse zusammen.

ID	Anforderung	Status
FA1	Identifikation von Services	✓
FA2	Aggregierte Serviceübersicht	✓
FA3	Navigation zu Artefakten	✓
FA4	Darstellung von Beziehungen	●
NFA1	VS Code Integration	✓
NFA2	CAP-Kompatibilität	✓
NFA3	Performance	✓
NFA4	Automatische Aktualisierung	✗

✓ erfüllt ● teilweise erfüllt ✗ nicht erfüllt

Tabelle 9 — Übersicht der Anforderungserfüllung

7.4 Explorative Entwickler-Rückmeldungen

Ergänzend zur funktionalen Evaluation wurden explorative Rückmeldungen von zwei CAP-Entwicklern eingeholt (Details in Anhang C, Tabelle 12). Die Teilnehmer testeten die Extension an unterschiedlichen Projekten (bookshop Sample vs. produktives System mit 131 Entities) und bestätigten die Benutzerfreundlichkeit (5/5).

Die Navigationsaufwand-Reduktion wurde mit 3/5 bzw. 4/5 bewertet. Die Rückmeldungen identifizierten Verbesserungspotenziale (Sortierung bei großen Projekten, extend-Statement-Duplikate) und bestätigten die in NFA4 identifizierte Limitation der fehlenden automatischen Aktualisierung.

7.5 Qualitative Beobachtungen

Neben den quantitativen Metriken wurden qualitative Aspekte der Benutzererfahrung beobachtet.

Die servicezentrierte Strukturierung ist intuitiv erfassbar. Die Trennung zwischen Inbound- und Outbound-Services folgt der fachlichen Logik und erleichtert die Orientierung in Projekten mit externen API-Abhängigkeiten. Die hierarchische Gruppierung (Services → Entities → Fields) entspricht dem mentalen Modell von CAP-Entwicklern, die in fachlichen Services denken, nicht in technischen Ordern.

Die automatische Expansion beim Filtern reduziert manuelle Interaktionen. Treffer sind sofort sichtbar, ohne dass der User mehrere Ordner manuell öffnen muss. Die Hervorhebung von Suchtreffern im Tree erleichtert das Scannen großer Ergebnislisten. Die Kombination von Freitext-Suche und strukturierten Filtern (@entity, @action) ermöglicht präzise Abfragen ohne komplexe Syntax.

Die Integration der LSP-Features (Go to Definition, Find References, Go to Implementation) im Context-Menü fügt sich nahtlos in den gewohnten VS Code-Workflow ein. Die Hover-Tooltips mit Typ-Informationen und Flags reduzieren die Notwendigkeit, Dateien zu öffnen, nur um Metadaten zu prüfen. Die automatische Entdeckung von Implementierungsdateien erspart manuelle Suchen nach Handler-Dateien mit unklaren Namenskonventionen.

Als Limitation wurde beobachtet, dass die fehlende Association-Darstellung (FA4) bei komplexen Datenmodellen mit vielen Beziehungen zu Orientierungsschwierigkeiten führen kann. Ein Entwickler muss manuell über Go to Definition navigieren, um zu erkennen, dass ein Feld auf eine andere Entity verweist. Die manuelle Re-

fresh-Notwendigkeit (NFA4) unterbricht den Arbeitsfluss bei iterativer Entwicklung, da Änderungen erst nach explizitem Reload sichtbar werden.

7.6 Einordnung der Ergebnisse

Die Evaluation fokussiert auf die funktionale Prüfung der Anforderungen. Eine empirische Nutzerstudie mit Zeitmessungen war gemäß der Abgrenzung in Kapitel 4 nicht Bestandteil dieser Arbeit. Die Szenarien zeigen dennoch eine klare Reduktion des Navigationsaufwands um durchschnittlich 79%.

Die drei Testprojekte decken unterschiedliche Komplexitätsstufen ab, von einem kleinen Referenzprojekt bis zu einem produktiven Enterprise-Anwendungs-System. Die Architektur durch die Nutzung des Language Servers lässt eine gute Skalierbarkeit auch für größere Projekte erwarten.

Die Ergebnisse zeigen, dass die Extension das identifizierte Navigationsproblem adressiert und sechs von acht Anforderungen vollständig erfüllt. Die nicht erfüllte automatische Aktualisierung sowie die fehlende Association-Darstellung bieten Ansatzpunkte für zukünftige Erweiterungen.

8 Diskussion und Fazit

Dieses Kapitel bildet die sechste Phase des DSR-Prozesses nach Peffers et al. [11]. Die Kommunikationsphase dient der Reflexion und Einordnung der Ergebnisse. Im Folgenden werden die Evaluationsergebnisse interpretiert, Limitationen benannt und Ansatzpunkte für zukünftige Arbeiten aufgezeigt.

8.1 Diskussion der Ergebnisse

Die Evaluation zeigt, dass die Extension das identifizierte Navigationsproblem adressiert. Die gemessene Reduktion des Navigationsaufwands um durchschnittlich 79% in den evaluierten Szenarien bestätigt die zentrale Annahme dieser Arbeit. Die technische Ordnerstruktur von CAP-Projekten erschwert das Verständnis einzelner Services, und eine servicezentrierte Sicht kann dieses Problem lösen.

Die Generalisierbarkeit dieser Ergebnisse ist durch die Stichprobengröße von drei Projekten limitiert. Eine Evaluation an einer größeren Projektmenge ($N > 10$) würde die Aussagekraft der 79%-Reduktion erhöhen und statistische Signifikanz ermöglichen.

Die Wahl der Language Server-basierten Integration hat sich als richtig erwiesen. Die Nutzwertanalyse in Kapitel 5 identifizierte diese Strategie als vorteilhafteste Option, und die Evaluation bestätigt diese Einschätzung. Der Language Server liefert präzise Positionsinformationen, die eine zeilengenaue Navigation ermöglichen. Die semantische Genauigkeit ist durch die Nutzung des CAP-Compilers garantiert. Ein eigener Parser hätte diesen Grad an Zuverlässigkeit nicht erreichen können.

8.2 Wissenschaftlicher Beitrag

Diese Arbeit liefert einen Beitrag in drei Bereichen.

Methodisch demonstriert die Arbeit die Anwendung des DSR-Ansatzes auf Navigationsprobleme in domänengetriebenen Frameworks. Die Nutzwertanalyse in Kapitel 5 vergleicht drei Integrationsstrategien anhand von neun gewichteten Kriterien. Diese systematische Entscheidungsfindung ist auf andere Werkzeugentwicklungen übertragbar, die zwischen Parser-basierter, Compiler-basierter und Language Server-basierter Integration wählen müssen.

Praktisch liefert das entwickelte Artefakt eine funktionierende VS Code Extension [3]. Der implementierte Beitrag zum CDS Language Server (x-Flag in workspace/symbol) [51] erweitert dessen API um servicezentrierte Metadaten. Diese Erweiterung ist permanent in den Language Server integriert und steht anderen Tools zur Verfügung.

Empirisch quantifiziert die Arbeit den Navigationsaufwand in CAP-Projekten anhand konkreter Szenarien. Die gemessene durchschnittliche Reduktion um 79% basiert auf reproduzierbaren Szenarien in drei Projekten unterschiedlicher Größe und liefert eine empirische Grundlage für zukünftige Untersuchungen zum Zusammenhang zwischen Projektgröße und Navigationskomplexität.

8.3 Limitationen

Die Extension erfüllt sechs von acht Anforderungen vollständig. Zwei Anforderungen sind nicht oder nur teilweise erfüllt.

Die automatische Aktualisierung der Artefaktübersicht ist nicht implementiert. Nach dem Speichern einer geänderten CDS-Datei muss der Entwickler den Tree manuell aktualisieren. Diese Einschränkung beeinträchtigt den Arbeitsfluss, ist aber technisch lösbar. Die VS Code API bietet File-Watcher-Events, die eine automatische Aktualisierung ermöglichen würden.

Die Darstellung referenzieller Associations zwischen Entities ist nicht umgesetzt. VS Code's TreeView API ist auf hierarchische Strukturen ausgelegt, während Associations Graph-Strukturen darstellen. Eine Visualisierung dieser Beziehungen würde

alternative UI-Patterns erfordern, die nicht Teil der TreeView-Architektur sind. Dies stellt ein architektonisches Limit der gewählten Darstellungsform dar.

Die Evaluation basiert auf drei Testprojekten und einer qualitativen Analyse der Navigationsszenarien. Die gemessene Reduktion um 79% ist reproduzierbar, aber bei einer Stichprobengröße von drei Projekten nicht statistisch generalisierbar. Eine empirische Nutzerstudie mit Zeitmessungen, größerer Projektanzahl und Entwickler-Befragungen könnte zusätzliche Erkenntnisse zur tatsächlichen Produktivitätssteigerung und statistischen Signifikanz liefern. Die Szenarien wurden manuell definiert und gezählt, was eine gewisse Subjektivität birgt. Eine automatisierte Metrik-Erfassung über User-Tracking würde objektivere Daten liefern.

8.4 Praktische Implikationen

Die Ergebnisse haben praktische Implikationen für CAP-Entwickler und Werkzeughersteller.

Für CAP-Entwickler zeigt die Evaluation, dass servicezentrierte Navigation den Navigationsaufwand messbar reduziert, wobei der Mehrwert mit der Projektgröße steigt. Die Ladezeiten von 0.56s bis 1.86s liegen unter der Wahrnehmungsschwelle, sodass die Extension den Arbeitsfluss nicht unterbricht.

Für Werkzeughersteller demonstriert die Arbeit, dass Language Server als Integrationsschicht eine tragfähige Architektur darstellen. Die Erweiterung des CDS Language Servers um das x-Flag zeigt, dass domänenspezifische Metadaten in LSP-konforme Strukturen eingebettet werden können, ohne die Standardkonformität zu verletzen. Die Extension nutzt ausschließlich Standard-LSP-Requests (`workspace/symbol`, `textDocument/hover`, `textDocument/implementation`), was die Wartbarkeit und Erweiterbarkeit sichert.

Die Arbeit zeigt, dass das CAP-Prinzip der Separation of Concerns und die Entwickler-Erfahrung nicht im Widerspruch stehen müssen. Die physische Trennung der Artefakte bleibt erhalten, während Werkzeuge eine logische Aggregation be-

reitstellen. Dies vermeidet einen Zielkonflikt zwischen Architekturprinzipien und Usability.

8.5 Ausblick

Die nicht erfüllten Anforderungen bieten Ansatzpunkte für zukünftige Erweiterungen.

Die Implementierung der automatischen Aktualisierung würde den Arbeitsfluss verbessern. Durch Registrierung auf File-Watcher-Events könnte die Extension Änderungen an CDS-Dateien erkennen und den Tree automatisch aktualisieren. Dies würde NFA4 vollständig erfüllen.

Die Visualisierung von Associations würde das Verständnis von Datenmodellen erleichtern. Da TreeView nur hierarchische Strukturen unterstützt, würde dies alternative UI-Patterns erfordern. Mögliche Ansätze wären Hover-Tooltips, die das Ziel einer Association anzeigen, oder klickbare Verweise, die direkt zur referenzierten Entity navigieren. Eine separate Graph-Ansicht für Datenmodell-Beziehungen wäre eine umfassendere Lösung, würde aber die Komplexität der Extension deutlich erhöhen. Die explorativen Entwickler-Rückmeldungen liefern zusätzliche Verbesserungsvorschläge: Sortierung nach Name bei großen Projekten (>100 Entities), Darstellung von Annotations über annotate-Statements, und Namespace-Gruppierung zur besseren Strukturierung bei verschachtelten Modellen.

Eine Nutzerstudie könnte die quantitativen Ergebnisse dieser Arbeit ergänzen. Messungen der tatsächlichen Zeitersparnis und Befragungen zur Zufriedenheit würden die praktische Relevanz der Extension validieren.

8.6 Reflexion der Forschungsmethode

Die Anwendung des DSR-Ansatzes nach Peffers et al. hat sich für diese Arbeit als geeignet erwiesen. Die strukturierte Progression durch die sechs Phasen gewährleistete eine systematische Entwicklung des Artefakts. Die Problemidentifikation in Phase 1 lieferte konkrete Szenarien mit quantifizierten Navigationsschritten. Die

Nutzwertanalyse in Phase 3 führte zu einer nachvollziehbaren Strategiewahl. Die Evaluation in Phase 5 validierte die Zielerreichung anhand der definierten Anforderungen.

Eine Limitation ist die einmalige Iteration zwischen Build und Evaluate. Mehrere Build-Evaluate-Zyklen hätten möglicherweise zusätzliche Verbesserungspotenziale identifiziert. Die Evaluation in Phase 5 identifizierte Limitationen (automatische Aktualisierung, Association-Darstellung), die in einer zweiten Iteration hätten adressiert werden können. Der Zeitrahmen einer Bachelorarbeit begrenzt jedoch die Anzahl möglicher Iterationen. Peffers et al. beschreiben DSR explizit als iterativen Prozess [11], diese Arbeit realisiert jedoch nur einen Durchlauf.

Die Evaluation basiert auf Szenarien und qualitativen Beobachtungen, nicht auf empirischen Nutzerstudien mit Entwicklern. Die gemessene Reduktion von 79% ist reproduzierbar, aber nicht durch Zeitmessungen mit Probanden validiert. Eine partizipative DSR-Methode mit stärkerer Nutzerbeteiligung hätte zusätzliche Erkenntnisse zur Akzeptanz und tatsächlichen Nutzung liefern können. Dies war jedoch gemäß der Abgrenzung in Kapitel 4 nicht Bestandteil dieser Arbeit.

8.7 Fazit

Diese Arbeit entwickelt und evaluiert eine VS Code Extension zur servicezentrierten Navigation in SAP CAP-Projekten. Das Ausgangsproblem ist die Verteilung servicebezogener Artefakte über mehrere Dateien und Verzeichnisse, die das Verständnis einzelner Services erschwert.

Die Arbeit folgt dem DSR Ansatz nach Peffers et al. und durchläuft alle sechs Phasen des Prozessmodells. Die Anforderungsanalyse identifiziert das Navigationsproblem und quantifiziert den Aufwand anhand konkreter Szenarien. Der Vergleich dreier Integrationsstrategien mittels Nutzwertanalyse führt zur Auswahl der Language Server-basierten Integration. Die Implementierung setzt diese Strategie als VS Code Extension um.

Die Evaluation an drei Testprojekten unterschiedlicher Größe zeigt eine durchschnittliche Reduktion des Navigationsaufwands um 79%. Sechs von acht Anforderungen werden vollständig erfüllt. Die Extension aggregiert alle servicebezogenen Artefakte in einer hierarchischen Ansicht und ermöglicht präzise Navigation zu Quellcode-Positionen.

Das entwickelte Artefakt löst das identifizierte Problem und bietet einen konkreten Mehrwert für CAP-Entwickler. Die servicezentrierte Sicht ergänzt die dateibasierte Navigation und reduziert den kognitiven Aufwand beim Verstehen und Bearbeiten von Services.

Literatur

- [1] SAP, „Concepts Overview | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/get-started/concepts>
- [2] SAP, „Core Data Services (CDS) | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/cds/>
- [3] SAP, „cap/vscode-cds-explorer: VSCode CDS Project Explorer (experimental)“. Zugegriffen: 10. Mai 2026. [Online]. Verfügbar unter: <https://github.tools.sap/cap/vscode-cds-explorer>
- [4] SAP, „Domain Modeling | capire“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/guides/domain>
- [5] SAP, „Authorization | capire“. Zugegriffen: 26. April 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/guides/security/authorization>
- [6] capire, „GitHub – capire/bookshop: Our primer sample to get started in a nutshell“. Zugegriffen: 25. Februar 2026. [Online]. Verfügbar unter: <https://github.com/capire/bookshop>
- [7] SAP, „The Bookshop Sample – Separation of Concerns | capire“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/get-started/bookshop#separation-of-concerns>
- [8] O. Nierstrasz und F. Acher mann, „Separation of Concerns through Unification of Concepts“, in *ECOOP 2000 Workshop on Aspects & Dimensions of Concerns*, Bern, Switzerland, 2000.
- [9] Microsoft, „Code Navigation | Visual Studio Code“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/docs/editing/editingevolved#_go-to-definition

- [10] Microsoft, „User Interface | Visual Studio Code“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/docs/getstarted/userinterface#_outline-view
- [11] K. Peffers, T. Tuunanen, M. A. Rothenberger, und S. Chatterjee, „A Design Science Research Methodology for Information Systems Research“, *Journal of Management Information Systems*, Bd. 24, Nr. 3, S. 45–77, 2007, doi: [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
- [12] A. R. Hevner, S. T. March, J. Park, und S. Ram, „Design Science in Information Systems Research“, *MIS Quarterly*, Bd. 28, Nr. 1, S. 75–105, 2004, [Online]. Verfügbar unter: <https://www.jstor.org/stable/25148625>
- [13] SAP, „About CAP | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/>
- [14] C. Fehling, „Cloud Computing Patterns: Identification, Design, and Application“, Dissertation, Stuttgart, Germany, 2014. [Online]. Verfügbar unter: <https://elib.uni-stuttgart.de/handle/11682/8568>
- [15] R. Kumar und P. Sharma, „Cloud Native Application Development: Best Practices and Challenges“, *International Journal of Research Publication and Reviews*, Bd. 5, Nr. 3, S. 3674–3681, 2024.
- [16] M. P. Papazoglou, „Service-Oriented Computing: Concepts, Characteristics and Directions“, in *Proceedings of the Fourth International Conference on Web Information Systems Engineering (WISE)*, IEEE, 2003, S. 3–12. doi: [10.1109/WISE.2003.1254461](https://doi.org/10.1109/WISE.2003.1254461).
- [17] OASIS, „OData Version 4.01“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://www.odata.org/documentation/>
- [18] OpenAPI Initiative, „OpenAPI Specification, Version 3.2.0“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://spec.openapis.org/oas/latest.html>

- [19] R. Krebs, C. Momm, und S. Kounev, „Architectural Concerns in Multi-Tenant SaaS Applications“, in *Proceedings of the 2nd International Conference on Cloud Computing and Services Science (CLOSER)*, Porto, Portugal: SciTePress, 2012, S. 426–431.
- [20] SAP, „Definition Language (CDL) | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/cds/cdl>
- [21] SAP, „Core Schema Notation (CSN) | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/cds/csn>
- [22] Sourcegraph, „Langserver.org – A community-driven source of knowledge for Language Server Protocol implementations“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://langserver.org/>
- [23] D. Bork und P. Langer, „An Introduction to the Protocol, its Use, and Adoption for Web Modeling Tools“, *Enterprise Modelling and Information Systems Architectures*, Bd. 18, Nr. 9, 2023, doi: [10.18417/emisa.18.9](https://doi.org/10.18417/emisa.18.9).
- [24] Microsoft, „Language Server Protocol Specification“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>
- [25] JSON-RPC Working Group, „JSON-RPC 2.0 Specification“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: <https://www.jsonrpc.org/specification>
- [26] J. Loth, C. Sundermann, T. Schrull, T. Brugger, F. Rieg, und T. Thüm, „UVLS: A Language Server Protocol For UVL“, in *Proceedings of the 27th ACM International Systems and Software Product Line Conference (SPLC '23) – Volume B*, Tokyo, Japan: ACM, Aug. 2023. doi: [10.1145/3579028.3609014](https://doi.org/10.1145/3579028.3609014).
- [27] Microsoft, „Extension API | Visual Studio Code“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api>

- [28] Microsoft, „Extension Host | Visual Studio Code Extension API“. Zugegriffen: 26. April 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api/advanced-topics/extension-host>
- [29] Microsoft, „Extension Capabilities Overview | Visual Studio Code Extension API“. Zugegriffen: 13. April 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api/extension-capabilities/overview>
- [30] Microsoft, „Tree View API | Visual Studio Code Extension API“. Zugegriffen: 13. April 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api/extension-guides/tree-view>
- [31] Microsoft, „Tree Data Provider | Tree View API“. Zugegriffen: 27. April 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api/extension-guides/tree-view#tree-data-provider>
- [32] Microsoft, „Updating Tree View Content | Tree View API“. Zugegriffen: 27. April 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api/extension-guides/tree-view#updating-tree-view-content>
- [33] Microsoft, „Webview API | Visual Studio Code Extension API“. Zugegriffen: 11. Mai 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api/extension-guides/webview>
- [34] Microsoft, „Language Server Extension Guide | Visual Studio Code Extension API“. Zugegriffen: 13. April 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/api/language-extensions/language-server-extension-guide>
- [35] J. Sweller, „Cognitive Load During Problem Solving: Effects on Learning“, *Cognitive Science*, Bd. 12, Nr. 2, S. 257–285, 1988, doi: [10.1207/s15516709cog1202_4](https://doi.org/10.1207/s15516709cog1202_4).
- [36] A. J. Ko, R. DeLine, und G. Venolia, „Information Needs in Collocated Software Development Teams“, in *Proceedings of the 29th International Conference on*

- Software Engineering (ICSE '07)*, IEEE Computer Society, Mai 2007, S. 344–353.
doi: [10.1109/ICSE.2007.45](https://doi.org/10.1109/ICSE.2007.45).
- [37] T. D. LaToza, G. Venolia, und R. DeLine, „Maintaining Mental Models: A Study of Developer Work Habits“, in *Proceedings of the 28th International Conference on Software Engineering (ICSE '06)*, Shanghai, China: ACM, Mai 2006, S. 492–501.
- [38] R. L. Baskerville und A. T. Wood-Harper, „A Critical Perspective on Action Research as a Method for Information Systems Research“, *Journal of Information Technology*, Bd. 11, Nr. 3, S. 235–246, 1996, doi: [10.1080/026839696345289](https://doi.org/10.1080/026839696345289).
- [39] P. Runeson und M. Höst, „Guidelines for Conducting and Reporting Case Study Research in Software Engineering“, *Empirical Software Engineering*, Bd. 14, Nr. 2, S. 131–164, 2009, doi: [10.1007/s10664-008-9102-8](https://doi.org/10.1007/s10664-008-9102-8).
- [40] A. Dresch, D. P. Lacerda, und J. A. V. Antunes Jr., *Design Science Research: A Method for Science and Technology Advancement*. Springer, 2014. doi: [10.1007/978-3-319-07374-3](https://doi.org/10.1007/978-3-319-07374-3).
- [41] SAP, „The Bookshop Sample – Focus on Domain | capire“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: <https://cap.cloud.sap/docs/get-started/bookshop#focus-on-domain>
- [42] O. Özkan, O. Babur, und M. van den Brand, „Domain-Driven Design in Software Development: A Systematic Literature Review on Implementation, Challenges, and Effectiveness“, in *Proceedings of the ACM Conference*, Nov. 2000, S. 1–29.
- [43] P. Tarr, W. Harrison, H. Ossher, A. Finkelstein, B. Nuseibeh, und D. Perry, „Workshop on Multi-Dimensional Separation of Concerns in Software Engineering“, in *Proceedings of the 22nd International Conference on Software Engineering (ICSE)*, Limerick, Ireland: ACM, 2000, S. 809–810.

- [44] M. M. Lehman, „Programs, Life Cycles, and Laws of Software Evolution“, *Proceedings of the IEEE*, Bd. 68, Nr. 9, S. 1060–1076, 1980, doi: [10.1109/PROC.1980.11805](https://doi.org/10.1109/PROC.1980.11805).
- [45] SAP, „cap/cds-lsp: Language Server for CDS“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: <https://github.tools.sap/cap/cds-lsp>
- [46] J. Nielsen, *Usability Engineering*. San Francisco, CA: Morgan Kaufmann, 1993.
- [47] A. V. Aho, M. S. Lam, R. Sethi, und J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2. Aufl. Boston: Addison-Wesley, 2007.
- [48] U. Götze, *Investitionsrechnung: Modelle und Analysen zur Beurteilung von Investitionsvorhaben*, 7. Aufl. Berlin, Heidelberg: Springer, 2014. doi: [10.1007/978-3-642-54622-8](https://doi.org/10.1007/978-3-642-54622-8).
- [49] P. Clements u. a., „Documenting Software Architectures: Views and Beyond“, Pittsburgh, PA, Special Report CMU/SEI-2000-SR-004, 2000.
- [50] npm, Inc., „About npm“. Zugegriffen: 11. Mai 2026. [Online]. Verfügbar unter: <https://docs.npmjs.com/about-npm>
- [51] SAP CAP Language Server Team, „CDS Language Server Release 9.9.0“. Zugegriffen: 27. April 2026. [Online]. Verfügbar unter: <https://github.tools.sap/cap/cds-lsp/releases/tag/rel%2F9.9.0>
- [52] capire, „GitHub – capire/xtravels: Travel Management System“. Zugegriffen: 27. April 2026. [Online]. Verfügbar unter: <https://github.com/capire/xtravels>
- [53] SAP, „ctsm-develop: Clinical Trial System“. Zugegriffen: 27. April 2026. [Online]. Verfügbar unter: <https://github.wdf.sap.corp/ctsm/ctsm.git>

Anhang

Testumgebung

Die Performance-Messungen in Kapitel 7 wurden unter folgenden Bedingungen durchgeführt:

Komponente	Spezifikation
Hardware	MacBook Air M2 (2022), 8 GB RAM
Betriebssystem	macOS Sequoia 15.3.0 (Darwin arm64 25.3.0)
VS Code	Version 1.110.0 (Universal)
Node.js	Version 22.22.0
Electron	Version 39.6.0
Chromium	Version 142.0.7444.265

Tabelle 10 — Testumgebung für Performance-Messungen

Anhang A: LSP workspace/symbol Response-Struktur

Die folgende Abbildung zeigt die Response-Struktur des workspace/symbol Requests mit dem /gax-Flag für die Domain-Model-Entity Travels aus db/schema.cds.

```
{
  „name“: „Travels“,
  „kind“: 5,
  „location“: {
    „uri“: „file://workspace/db/schema.cds“,
    „range“: {
      „start“: { „line“: 12, „character“: 7 },
      „end“: { „line“: 12, „character“: 14 }
    }
  },
  „containerName“: „sap.capire.travels“,
  „data“: {
    „flags“: 131072
  }
}
```

Quellcode 6 — LSP-Response für ein Symbol aus der /gax Query

Anhang B: Flag-Spezifikation für servicezentrierte Navigation

Die folgende Tabelle dokumentiert die Flags, die im `flags`-Feld der LSP-Response kodiert sind. Die Flags werden als Bitmasken implementiert und ermöglichen die Kategorisierung und Filterung von Symbolen.

Jedes Flag entspricht einer Zweierpotenz (Bit-Position). Der numerische Wert wird durch Potenzierung berechnet. Bei kombinierten Flags werden die Werte addiert:

- **Einzelles Flag:** DOMAIN_MODEL ist an Bit-Position 17 definiert. Der Wert berechnet sich als $2^{17} = 131072$.
- **Kombinierte Flags:** Wenn ein Symbol sowohl ASSOCIATION (Bit 12) als auch DEFINITION (Bit 0) hat, werden die Werte addiert: $2^{12} + 2^0 = 4096 + 1 = 4097$.

Flag	Bit	Bedeutung
DEFINITION	0	Kennzeichnet eine Definition (nicht nur Referenz)
REFERENCE	1	Kennzeichnet eine Referenz auf ein Symbol
ANNOTATION	2	Markiert Annotationen
EXPLICIT_NAMESPACE	3	Explizit deklarierter Namespace
IMPLICIT_NAMESPACE	4	Implizit abgeleiteter Namespace
TRANSLATION_STRING	5	Übersetzungsschlüssel für i18n
USING_PATH	6	Import-Pfad in using-Statement
EXTEND	7	Element aus extend-Statement
ASPECT	8	Markiert Aspects
TEXTS	9	Lokalisierte Texte
HAS_AUTOEXPOSED_ENTITIES	10	Service hat automatisch exponierte Entities
ANNOTATION_MODELER_XREF	11	Cross-Referenz für Annotation Modeler
ASSOCIATION	12	Association-Beziehung zwischen Entities
COMPOSITION	13	Composition-Beziehung zwischen Entities
FEDERATED	14	Federated Entity aus externem Service
CODE_LIST	15	Code-Liste für Value-Help
EXTERNAL_SERVICE	16	Externer Service (Remote API)
DOMAIN_MODEL	17	Entity im Datenbank-Schema ohne Service-Zugehörigkeit
AUTOEXPOSED_ENTITY	18	Automatisch exponierte Entity
ACTION	19	Action (modifiziert Daten)
VIEW	20	Projektion statt persistente Entity
ANNOTATED_IDENTIFIER	21	Identifier mit Annotations

Tabelle 11 — Flags für servicezentrierte Navigation

Anhang C: Explorative Entwickler-Rückmeldungen

Im Rahmen der Evaluation wurden explorative Rückmeldungen von zwei CAP-Entwicklern eingeholt.

Kriterium	Teilnehmer 1	Teilnehmer 2
CAP-Erfahrung	>5 Jahre	1-2 Jahre
Testprojekt	bookshop Sample	Produktives Projekt (131 Entities)
Gewählte Aufgabe	Finde alle Entities von CatalogService	Handler-Implementierung zu Service finden
Benutzerfreundlichkeit	5/5	5/5
Navigationsaufwand-Reduktion	3/5	4/5
Beziehungsdarstellung	5/5	5/5
Nutzungsabsicht	5/5	4/5
Positiv	TreeView-Übersicht funktioniert	• TreeView-Übersicht und Suche funktionieren gut • Direkte Navigation zur Definition
Probleme	• Kein Icon in Extension Bar • Icons im README fehlen • Buttons wirken zu dominant	• Suche schwer zu finden • Nach Refresh Zustand nicht erhalten • extend-Statements erscheinen doppelt • Java Action-Implementierung nicht gefunden
Verbesserungsvorschläge	Buttons in Titelbar integrieren	• Sortierung nach Name (bei >100 Entities) • Annotations über annotate anzeigen • Felder auch für Domain-Model anzeigen • Namespace-Gruppierung

Tabelle 12 — Übersicht der explorativen Entwickler-Rückmeldungen

Begründung der Nutzwertanalyse

Die folgende Tabelle dokumentiert die Begründung der Punktvergabe für die in Kapitel 5 durchgeführte Nutzwertanalyse der drei Integrationsstrategien.

Kriterium	Strategie	Punkte	Begründung
K1: Semantische Genauigkeit	Repository	2	Eigenes Parsing ohne Nutzung des CAP-Compilers; Risiko semantischer Fehlinterpretationen
	CSN	5	Nutzt kompiliertes CAP-Modell; semantisch korrekte Interpretation garantiert
	Language Server	5	Nutzt CAP-Compiler; semantische Analyse durch SAP-Tooling
K2: Positionsinformationen	Repository	3	Positionen aus eigenem Parsing verfügbar, aber aufwändige Implementierung
	CSN	1	CSN enthält keine Quellcode-Positionen (Zeile/Spalte)
	Language Server	5	Location-API liefert präzise Positionen
K3: VS Code Integration	Repository	2	Keine native Integration; eigene Adapter erforderlich
	CSN	3	JSON-Daten müssen transformiert werden
	Language Server	5	Native LSP-Schnittstellen direkt nutzbar
K4: Implementierungsaufwand	Repository	2	Hoher Aufwand: eigener Parser erforderlich
	CSN	4	Geringer Aufwand: JSON-Parsing
	Language Server	3	Mittlerer Aufwand: LSP-Client-Integration
K5: Wartbarkeit	Repository	2	Parser muss bei Sprachänderungen angepasst werden
	CSN	3	CSN-Schema kann sich ändern
	Language Server	4	LSP ist stabiler Standard
K6: Unabhängigkeit	Repository	5	Keine externen Abhängigkeiten
	CSN	2	Abhängigkeit vom CAP-Compiler
	Language Server	2	Abhängigkeit vom Language Server

Tabelle 13 — Detaillierte Begründung der Punktvergabe in der Nutzwertanalyse

Detaillierte Navigationsschritte für Bookshop-Szenarien

Die folgenden Tabellen dokumentieren die detaillierten Navigationsschritte für drei weitere Entwicklungsszenarien im Bookshop-Projekt. Das Szenario „Service verstehen“ ist in Kapitel 4 dargestellt.

#	Datei	Zeile	Aktion
1	db/schema.cds	4–13	Entity Books finden, Feld hinzufügen
2	srv/cat-service.cds	6–16	Projektionen ListOfBooks und Books anpassen
3	srv/admin-service.cds	5	AdminService Projection prüfen
4	srv/admin-constraints.cds	4–18	Validierungs-Constraints prüfen
5	srv/cat-service.js	9–23	CatalogService Handler prüfen
6	srv/admin-service.js	4–5	AdminService Handler prüfen

Tabelle 14 — Szenario „Feld hinzufügen“ – 6 Dateien, 6 Navigationsschritte

#	Datei	Zeile	Aktion
1	srv/cat-service.cds	3	Service-Definition identifizieren
2	srv/cat-service.cds	18–19	@requires-Annotation und betroffene Action
3	db/schema.cds	4–29	Exponierte Daten auf Sensibilität prüfen
4	srv/cat-service.js	14–23	Handler auf Auth-abhängige Logik prüfen

Tabelle 15 — Szenario „Auth anpassen“ – 3 Dateien, 4 Navigationsschritte

#	Datei	Zeile	Aktion
1	srv/cat-service.js	14	submitOrder-Handler finden
2	srv/cat-service.js	15–22	Handler-Logik analysieren
3	srv/cat-service.cds	19	Action-Definition und Parameter prüfen
4	db/schema.cds	4–13	Entity Books mit stock-Feld verstehen

Tabelle 16 — Szenario „Handler debuggen“ – 3 Dateien, 4 Navigationsschritte

Navigationsabläufe in CAP-Projekten

Die folgenden Tabellen dokumentieren die Navigationsschritte für typische Entwicklungsszenarien in verschiedenen CAP-Projekten. Für jedes Szenario werden die Abläufe ohne und mit Extension gegenübergestellt, um die Reduktion des Navigationsaufwands zu quantifizieren.

Szenario 1: Service verstehen

Aufgabe: Ein Entwickler soll sich einen Überblick über einen Service verschaffen und dessen exponierte Entities, Operationen und Struktur verstehen.

xtravels (TravelService)

Ohne Extension

#	Datei	Zeile	Zweck
1a	srv/travel-service.cds	5–22	Datei im Explorer öffnen
1b	srv/travel-service.cds	5–22	Zur Service-Definition navigieren
1c	srv/travel-service.cds	–	Zurück zur vorherigen Datei
2a	srv/travel-exports.cds	5–24	Datei im Explorer öffnen
2b	srv/travel-exports.cds	5–24	Export-Funktionen navigieren
2c	srv/travel-exports.cds	–	Zurück zur vorherigen Datei
3a	db/schema.cds	8–60	Datei im Explorer öffnen
3b	db/schema.cds	8–60	Domain-Entities navigieren
3c	db/schema.cds	–	Zurück zur vorherigen Datei
4a	srv/travel-service.js	3–12	Datei im Explorer öffnen
4b	srv/travel-service.js	3–12	Handler-Klasse navigieren
4c	srv/travel-service.js	–	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Klick: TravelService	Service-Definition öffnet sich
2	Klick: Entities (7)	Alle exponierten Entities sichtbar
3	Klick: Travels → Actions (5)	Alle Actions sichtbar

Tabelle 17 — Navigationsablauf „Service verstehen“ (xtravels) – Vergleich

bookshop (CatalogService)**Ohne Extension**

#	Datei	Zeile	Zweck
1a	srv/cat-service.cds	3-20	Datei im Explorer öffnen
1b	srv/cat-service.cds	3-20	Zur Service-Definition navigieren
1c	srv/cat-service.cds	-	Zurück zur vorherigen Datei
2a	db/schema.cds	4-29	Datei im Explorer öffnen
2b	db/schema.cds	4-29	Domain-Entities navigieren
2c	db/schema.cds	-	Zurück zur vorherigen Datei
3a	srv/cat-service.js	3-27	Datei im Explorer öffnen
3b	srv/cat-service.js	3-27	Handler-Implementierung navigieren
3c	srv/cat-service.js	-	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Klick: CatalogService	Service-Definition öffnet sich
2	Klick: Entities (2)	ListOfBooks und Books sichtbar

Tabelle 18 — Navigationsablauf „Service verstehen“ (bookshop) – Vergleich

ctsm-develop (StudyOverviewService)

Ohne Extension

#	Datei	Zeile	Zweck
1a	srv/models/ studyoverview-service.cds	11-186	Datei im Explorer öffnen
1b	srv/models/ studyoverview-service.cds	11-186	Zur Service-Definition navigieren
1c	srv/models/ studyoverview-service.cds	-	Zurück zur vorherigen Datei
2a	db/models/Forecast/...	-	Ordner im Explorer öffnen
2b	db/models/Forecast/...	-	Domain-Entities in Unterordnern suchen
2c	db/models/Forecast/...	-	Zurück zur vorherigen Datei
3a	srv/impl/studyoverview-service.js	1-60	Datei im Explorer öffnen
3b	srv/impl/studyoverview-service.js	1-60	Handler-Implementierung navigieren
3c	srv/impl/studyoverview-service.js	-	Zurück zur vorherigen Datei
4a	srv/models/external/*.csn	-	Ordner im Explorer öffnen
4b	srv/models/external/*.csn	-	Externe Service-Referenzen navigieren
4c	srv/models/external/*.csn	-	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Klick: StudyOverviewService	Service-Definition öffnet sich
2	Klick: Entities (19)	Alle exponierten Entities sichtbar

Tabelle 19 — Navigationsablauf „Service verstehen“ (ctsm-develop) – Vergleich

Szenario 2: Action lokalisieren

Aufgabe: Ein Entwickler soll eine bestimmte Action finden, ihre Definition verstehen und die Handler-Implementierung lokalisieren.

xtravels (acceptTravel)

Ohne Extension

#	Datei	Zeile	Zweck
1a	srv/travel-service.cds	9	Datei im Explorer öffnen
1b	srv/travel-service.cds	9	Action-Definition navigieren
1c	srv/travel-service.cds	–	Zurück zur vorherigen Datei
2a	srv/travel-service.js	142–161	Datei im Explorer öffnen
2b	srv/travel-service.js	142–161	Handler-Methode navigieren
2c	srv/travel-service.js	–	Zurück zur vorherigen Datei
3a	db/schema.cds	52–60	Datei im Explorer öffnen
3b	db/schema.cds	52–60	TravelStatus-Entity navigieren
3c	db/schema.cds	–	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Suchfeld: „acceptTravel“	Tree filtert auf Action
2	Klick: acceptTravel()	Datei öffnet sich an Zeile 9

Tabelle 20 — Navigationsablauf „Action lokalisieren“ (xtravels) – Vergleich

bookshop (submitOrder)**Ohne Extension**

#	Datei	Zeile	Zweck
1a	srv/cat-service.cds	19	Datei im Explorer öffnen
1b	srv/cat-service.cds	19	Action-Definition navigieren
1c	srv/cat-service.cds	–	Zurück zur vorherigen Datei
2a	srv/cat-service.js	14–23	Datei im Explorer öffnen
2b	srv/cat-service.js	14–23	Handler-Methode navigieren
2c	srv/cat-service.js	–	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Suchfeld: „submitOrder“	Tree filtert auf Action
2	Klick: submitOrder()	Datei öffnet sich an Zeile 19

Tabelle 21 — Navigationsablauf „Action lokalisieren“ (bookshop) – Vergleich

ctsm-develop (Entity Projection lokalisieren)

Ohne Extension

#	Datei	Zeile	Zweck
1a	srv/models/ studyoverview-service.cds	12	Datei im Explorer öffnen
1b	srv/models/ studyoverview-service.cds	12	Entity-Definition navigieren
1c	srv/models/ studyoverview-service.cds	–	Zurück zur vorherigen Datei
2a	db/models/...	–	Ordner im Explorer öffnen
2b	db/models/...	–	Suche nach zugrunde liegender Entity
2c	db/models/...	–	Zurück zur vorherigen Datei
3a	srv/impl/studyoverview-service.js	–	Datei im Explorer öffnen
3b	srv/impl/studyoverview-service.js	–	Suche nach Handler-Logik
3c	srv/impl/studyoverview-service.js	–	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Suchfeld: „StudyOverviewFilterBar“	Tree filtert auf Entity
2	Klick: StudyOverviewFilterBar	Datei öffnet sich an Zeile 12

Tabelle 22 — Navigationsablauf „Entity lokalisieren“ (ctsm-develop) – Vergleich

Szenario 3: Exponierte Entities identifizieren

Aufgabe: Ein Entwickler soll prüfen, welche Entities über die API exponiert sind, um Authorization-Regeln zu verstehen oder anzupassen.

xtravels (TravelService)

Ohne Extension

#	Datei	Zeile	Zweck
1a	Projektweite Suche	–	Suche nach @restrict starten
1b	Projektweite Suche	–	Ergebnisliste durchsehen
1c	Projektweite Suche	–	Zurück zur vorherigen Datei
2a	srv/travel-access-control.cds	3–8	Datei im Explorer öffnen
2b	srv/travel-access-control.cds	3–8	@restrict-Annotation navigieren
2c	srv/travel-access-control.cds	–	Zurück zur vorherigen Datei
3a	srv/travel-service.cds	7–20	Datei im Explorer öffnen
3b	srv/travel-service.cds	7–20	Betroffene Entities navigieren
3c	srv/travel-service.cds	–	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Klick: TravelService → Entities	Exponierte Entities sichtbar
2	Vergleich mit Domain Model	Trennung Service vs. Domain erkennbar

Tabelle 23 — Navigationsablauf „Exponierte Entities identifizieren“ (xtravels) – Vergleich

bookshop (CatalogService)**Ohne Extension**

#	Datei	Zeile	Zweck
1a	srv/cat-service.cds	6–16	Datei im Explorer öffnen
1b	srv/cat-service.cds	6–16	Exponierte Entities navigieren
1c	srv/cat-service.cds	–	Zurück zur vorherigen Datei
2a	db/schema.cds	4–29	Datei im Explorer öffnen
2b	db/schema.cds	4–29	Domain-Model zum Vergleich navigieren
2c	db/schema.cds	–	Zurück zur vorherigen Datei

Mit Extension

#	Aktion	Ergebnis
1	Klick: CatalogService → Entities	ListOfBooks und Books sichtbar

Tabelle 24 — Navigationsablauf „Exponierte Entities identifizieren“ (bookshop) – Vergleich

ctsm-develop (StudyOverviewService)

Ohne Extension

#	Datei	Zeile	Zweck
1a	srv/models/ studyoverview-service.cds	11-186	Datei im Explorer öffnen
1b	srv/models/ studyoverview-service.cds	11-186	19 exponierte Entities navigieren
1c	srv/models/ studyoverview-service.cds	-	Zurück zur vorherigen Datei
2a	db/models/...	-	Ordner im Explorer öffnen
2b	db/models/...	-	Domain-Model in verteilten Ordnern suchen
2c	db/models/...	-	Zurück zur vorherigen Datei
3a	Manuelle Analyse	-	Service vs. Domain manuell vergleichen
3b	Manuelle Analyse	-	Zuordnung dokumentieren
3c	Manuelle Analyse	-	Zurück zur Ausgangsdatei

Mit Extension

#	Aktion	Ergebnis
1	Klick: StudyOverviewService → Entities	19 exponierte Entities auf einen Blick

Tabelle 25 — Navigationsablauf „Exponierte Entities identifizieren“ (ctsm-develop)
– Vergleich

Hilfsmittelangabe zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung von wissenschaftlichen Arbeiten**Persönliche Angaben**

Meinel, Max Nachname, Vorname	7864687 Matrikelnummer
Ulmenweg 55, 68167 Mannheim Anschrift	WWI23SEB Kurs
max.meinel@sap.com E-Mail	015735574466 Telefon- / Handynummer

Für das Modul

Semester 5 Modulbezeichnung / Semester
--

muss ich am

11.05.2026 Datum der Frist
--

folgende Prüfungsleistung erbringen:

- ☐ Projektarbeit I ☐ Projektarbeit II
- ☐ Seminararbeit ☒ Bachelorarbeit
- ☐ Sonstige

genaue Bezeichnung der sonstigen Prüfungsleistung
--

Zur Verwendung KI-gestützter Werkzeuge erkläre ich in Kenntnis des Hinweisblatts "Hinweise zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung wissenschaftlicher Arbeiten und die prüfungsrechtlichen Folgen ihres Einsatzes" Folgendes:

- Ich habe mich aktiv über die Leistungsfähigkeit und Beschränkungen der in meiner Arbeit eingesetzten KI-Werkzeuge informiert.
- Bei der Anfertigung der Prüfungsleistung habe ich durchgehend eigenständig und beim Einsatz KI-gestützter Werkzeuge maßgeblich steuernd gearbeitet.
- Insbesondere habe ich die Inhalte entweder aus wissenschaftlichen oder anderen zugelassenen Quellen entnommen und diese gekennzeichnet oder diese unter Anwendung wissenschaftlicher Methoden selbst entwickelt.
- Mir ist bewusst, dass ich als Autor/in der Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.
- Ich habe keine weiteren als die nachstehend von mir benannten KI-gestützten Werkzeuge zur Erstellung der Arbeit eingesetzt und diese nur in der angegebenen Art und Weise.

- Soweit ich KI-gestützte Werkzeuge zur Erstellung der Arbeit eingesetzt habe, gebe ich diese in der nachstehenden "Übersicht verwendeter Hilfsmittel" mit ihrem Produkt-namen und ihres genutzten Funktionsumfangs vollständig an; wörtliche oder sinn-gemäße Übernahmen KI-generierter Inhalte habe ich in ein separates Verzeichnis aufgenommen und im Text belegt (z. B. als Fußnote). Diese Inhalte sind als pdf-Datei in den elektronischen Beigaben hinterlegt.

Produktname(n) eingesetzter Hilfsmittel:

- ChatGPT 5.4
- Claude Opus 4.5; 4.6; 4.7
- Claude Code; GitHub CoPilot

Genutzter Funktionsumfang im Hinblick auf

- **Themenfassung und Strukturierung** (Aufgabenstellung oder Präzisierung, Forschungsansatz, Gliederung)

- Überblick über Themen bekommen
- Zusammenfassen sowie Übersetzen von Quellen
- Agentic Coding für die Erstellung der VS Code Extension (Claude Code; GitHub CoPilot)

- **Themenbearbeitung** (Darstellung/Beschreibung des Problems, Darlegung von wissenschaft-lichen Grundlagen, Schlussfolgerungen allgemein und für das konkrete Thema / Erkenntnisgewinn, Evaluierung des Textes / Feedback der KI)

- Plan für die Zeitliche Einteilung sowie ein Plan B (Chat GPT 5.4)
- zu Beginn der Arbeit einen Entwurf für die Gliederung erstellt (Chat GPT 5.4)
- mögliche Probleme für die Erstellung dieser Arbeit wurden abgewogen (Chat GPT 5.4)

- **Quellenrecherche, -auswahl und -auswertung** (welche Quellen wurden durch das KI-Werkzeug gefunden, wie erfolgte die weitere Recherche)

- zu Beginn der Arbeit Überblick an Quellen gesammelt
- Quellen wurden zusammengefasst, sowie übersetzt; Englisch zu Deutsch

- **Formale Gestaltung, insbesondere Sprache** (Welche Eingabe wurde an die KI getätigt? Textgenerierung, Textkorrektur, Paraphrasieren und Umschreiben, Übersetzen, kreatives Schreiben)

- Quelle wurde von Englisch zu Deutsch übersetzt, für bessere Verständlichkeit („Information Needs in Collocated Software Development Teams“; Literaturverzeichnis Eintrag 36)

Hinweis:

Geben Sie in der jeweiligen Rubrik die Seitenzahl in ihrer wissenschaftlichen Arbeit an und erläutern Sie die Art und Weise sowie den Umfang der von Ihnen genutzten KI-basierten Werkzeuge.

Münster (Hessen)
11.05.2026

Ort, Datum

Unterschrift der/des Studierenden